

第四篇 软件工程

"软件不是写出来的，是生长出来的。" —— 一个软件工程师的感悟

本篇导读

嘿！欢迎来到软件工程的世界！你是不是觉得"软件工程"这四个字听起来特别正经、特别枯燥、特别像那种让你在图书馆睡着的课？

大错特错！

软件工程是所有计算机科学中最"活"的一门。为什么？因为它是关于人的学问——一群人怎么合作，怎么把脑子里的想法变成能跑起来的代码，怎么不让项目变成一锅粥。

在这篇里，你会跟着**小红**一起从零开始创建她的创业公司。她会踩遍软件工程的每一个坑，而你，将站在她的肩膀上学习——不用亲身跳坑，也能学会怎么不跳坑。这叫什么？这叫**智慧**。

GEEK NOTE

软件工程在考试中约占**20分**。它不像离散数学那样需要死记硬背公式，但概念特别多、模型特别多、方法特别多。这篇内容的特点就是：**理解比死记更重要，联系比孤立更重要**。你会发现，瀑布模型、敏捷开发、UML图——这些东西在真实项目中是互相缠绕的，就像意大利面一样（不过我们希望你的代码不要像意大利面）。

第1章 软件工程概论：小红的创业项目刚立项，就遇到了第一个"敌人"

小红这辈子最骄傲的事情，是她大三时参加了一个黑客马拉松，用48小时写出了一个帮人找停车位的App。评委老师说："这个想法不错，有商业潜力。"

小红心想：**那还等什么？创业啊！**

她拉上了两个同学——小刚（擅长后端，Java一把梭）和小美（UI设计小能手），三个人在学校旁边的咖啡店里"注册"了他们的公司。"我们就做一个停车App吧，"小红说，"我两天就能写出来。"

三个月后.....

小红看着满屏的Bug，语重心长地说了一句流传千古的话：

"我当初以为写软件就是写代码。我错了。"

1.1 软件不只是代码

什么是软件？

来，我们做一个简单的测试。下面哪些是"软件"？

- A. 微信App
- B. 微信的用户使用手册
- C. 微信的数据库里存储的聊天记录
- D. 微信的需求文档
- E. 微信的测试用例

如果你的答案是"以上都是"，恭喜你——你已经比三个月前的小红聪明了。

软件不只是程序代码。软件 = 程序 + 数据 + 文档。

软件的三要素：

程序（Program）
— 可执行的代码，告诉计算机做什么

数据（Data）
— 程序运行所需要的数据结构和数据内容

文档（Document）
— 需求文档、设计文档、用户手册.....
— 小明最讨厌写，但又不能不写的东西

[Diagram: 一个三角形，三个顶点分别是"程序"、"数据"、"文档"，中间写着"软件"。旁边配字："三点支撑，缺一不可。"]

软件有什么特点？跟硬件比一下：

维度	硬件	软件
形态	可见可摸	逻辑实体，不可见
生产方式	制造（工厂线）	开发（人的智力活动）
磨损	会磨损老化	不会磨损，但会"腐化"
复用性	复用成本高	可复用，但需适配

WATCH IT!

软件不会像汽车轮胎那样"磨坏"，但它会**腐化（Software Decay）**。你改一个Bug，可能引入了两个新Bug。你加一个功能，导致三个老功能不工作了。这就像一间房子，你不停地装修，总有一天它会塌——如果不做合理的工程化管理的话。

这就是为什么我们需要**软件工程**。

软件危机：那个曾经让整个行业恐慌的词

1968年，在德国的一个小镇上，一群计算机科学家聚在一起开了一个会议。这个会议后来的影响堪比"遵义会议"——它正式宣告了"软件危机"的到来。

什么？软件还有危机？

是的。当时的场景是这样的：

- 项目预算一超再超，IBM的OS/360操作系统花了5亿美元
- 进度一再延误，有的项目延迟了好几年
- 软件质量差，Bug多到让人怀疑人生
- 维护成本高到离谱，改一行代码比新写一百行还贵
- 用户需求变了，开发团队居然不知道

这就是软件危机的五大表现。考试的时候你一定要记住这五个词：

预算超支、进度延误、质量低下、维护困难、需求不清

为什么会出现软件危机？

根本原因只有一个：**软件太复杂了，而人类的脑容量没有跟上。**

具体来说：

1. 软件是逻辑产品，不可见——你没法"看一眼"就知道它的进度
2. 软件开发高度依赖个人技能——大神写了没人看得懂
3. 需求在变——用户自己都不知道自己想要什么
4. 管理困难——没有科学的估算方法和进度控制

怎么解决？

软件工程应运而生。

RELAX

不要被"危机"两个字吓到。软件危机到今天也没有完全解决，但我们有了很多好工具、好方法。考试不会让你解决软件危机，只会考你软件危机的**表现、原因、解决途径**。记住上面那几个词就够了。

1.2 什么是软件工程？

软件工程的定义啊，不同的书说不同的版本，但核心就一句话：

软件工程是采用工程化的方法开发与维护软件的学科。

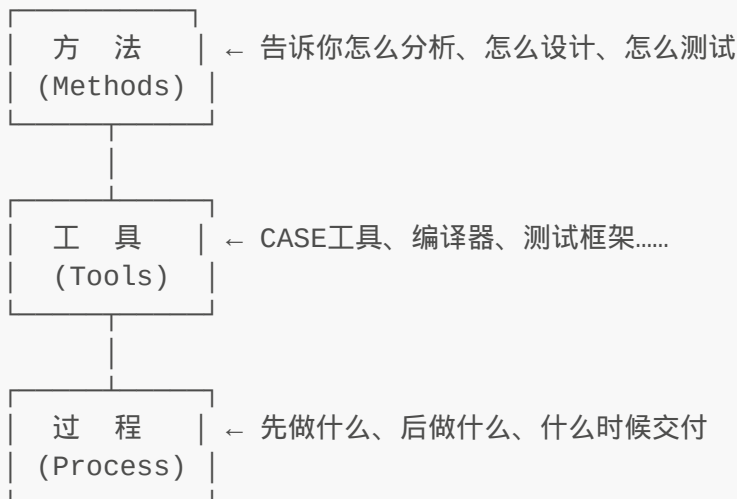
更具体一点，1968年NATO会议的定义是：

软件工程是建立和使用一套合理的工程原则，以便经济地获得可靠的、能在实际机器上高效运行的软件。

拆开来看，软件工程的**三个基本要素**（三要素）：

1. **方法 (Methods)** ——怎么做（技术层面，"How"）
2. **工具 (Tools)** ——用什么做（"With What"）

3. 过程 (Process) ——什么时候做什么 (管理层面, "When")



软件工程的目标是什么？四个词：

正确性、易用性、经济性、可维护性

考试会考"软件工程的目标不包括以下哪一项"，选项里经常混入一些看上去很对但其实不是目标的东西，比如"最高性能"——软件工程追求的是"足够好"，而不是"完美"。

软件工程的四大原则：

1. **严格性 (Rigor)** ——每一步都要有明确的规范
2. **关注点分离 (Separation of Concerns)** ——分而治之
3. **模块化 (Modularity)** ——像乐高积木一样搭软件
4. **抽象 (Abstraction)** ——忽略细节，抓住本质

SHARPEN YOUR PENCIL

小红的停车App项目出现了以下情况，请判断分别属于软件危机的哪种表现：

1. 原本计划3个月完成，结果做了6个月还没做完。_____
2. 用户反馈了一个Bug，小刚改了三天的，又引入了两个新Bug。_____
3. 当初预算10万，结果花了25万。_____
4. 用户说"我要一个能自动识别空车位的功能"，但小红当时以为用户只是要手动标记。_____
5. App上线后，用户每天投诉3次，闪退、卡顿、数据丢失....._____

(答案见本章末尾)

1.3 软件工程方法学

整个软件工程发展史，说白了就是两种大流派的"华山论剑"。

第一种：结构化方法 (Structured Methods)

诞生于20世纪70年代，核心思想是：**自顶向下，逐步求精，模块化**。

它看待软件的方式是：软件 = 数据结构 + 算法（数据和对数据的操作是分离的）。

代表方法：Yourdon结构化方法、SASD（结构化分析与结构化设计）

核心工具：

- 数据流图（DFD）——描述数据怎么流动
- 数据字典（DD）——定义每个数据元素
- 结构图（SC）——描述模块之间的调用关系

第二种：面向对象方法（Object-Oriented Methods）

诞生于20世纪80年代末，核心思想是：**软件 = 对象 + 消息**（数据和操作封装在一起）。

代表方法：Booch方法、OMT方法、OOSE方法（后来三合一成为UML）

核心工具：

- 用例图、类图、顺序图、状态图.....

下面这张表值得背下来：

维度	结构化方法	面向对象方法
核心概念	功能分解，数据与过程分离	对象封装，数据与操作合一
分析工具	DFD、DD、E-R图	用例图、类图
设计工具	SC图、流程图、N-S图	类图、顺序图、组件图
复用性	较差	较好（继承、多态）
适用场景	需求比较稳定的项目	需求变化频繁的项目
思维方式	怎么做（How）	是什么（What）

BRAIN POWER

如果小红用结构化方法来设计她的停车App，她可能会先画一张数据流图，描述"用户输入车位信息 → 系统匹配 → 返回结果"的数据流。

如果用面向对象方法，她可能会先定义一个ParkingSpot类（属性：位置、是否空闲；方法：reserve()、release()），再定义一个User类（属性：姓名、车牌号；方法：searchParking()、pay()）。

思考一下：如果App需要新增一个"预约停车场月卡"的功能，用哪种方法修改起来更方便？为什么？

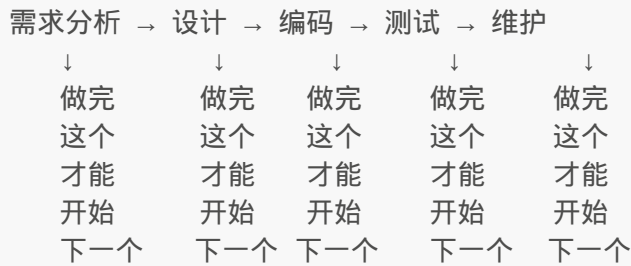
1.4 软件过程：从小作坊到流水线

软件开发不是随机乱敲键盘，它需要一个**过程**。

软件过程就是：**一系列活动的集合，这些活动把用户需求转化为软件产品**。

瀑布模型（Waterfall Model）

1970年，Winston Royce提出了这个影响深远的模型：



像瀑布一样，水只能从上往下流，**绝不回头**。

特点：

- 阶段间具有顺序性和依赖性——前一阶段完成后才能开始后一阶段
- 推迟实现——先想清楚再动手
- 每个阶段必须提交文档——文档驱动

优点：简单、易管理、里程碑清晰 **缺点：**需求必须一开始就完全确定（现实中几乎不可能），用户很久才能看到可运行的软件

适用场景：需求非常稳定、明确的项目（如航天软件、医疗设备软件）

小红一拍桌子："这不适合我们！用户需求三天两头变，瀑布模型会把我们淹死的。"

WATCH IT!

考试特别爱考瀑布模型的特点和优缺点。记住关键词："不可逆转"。瀑布模型最重要的特点就是阶段顺序执行，这在需求变化频繁的项目中是一个致命缺陷。

增量模型（Incremental Model）

小红说："那我不如先做一个能用的最简版本，然后一点一点往上加功能？"

对，这就是增量模型。

核心思想：把软件分成多个增量（构件），第一个增量是核心功能，后续增量逐步叠加。



优点：

- 用户可以尽快看到部分功能（不用等到全部完成）

- 需求变化可以逐步适应
- 核心功能先验证，降低风险

缺点：

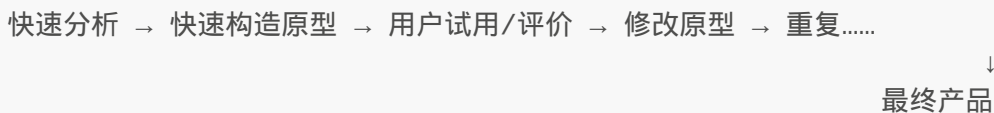
- 每次增量都不能破坏已有的系统结构（要求良好的架构设计）
- 用户可能把第一个增量当成"最终版本"而失望

原型模型 / 演化模型 (Prototyping / Evolutionary Model)

"用户自己都不知道自己想要什么"——这是软件工程的第一定律。

怎么办？给他一个原型 (Prototype) ！

原型模型的核心是：快速构建一个"能做样子的"版本，让用户玩一下，然后根据反馈再改。



两种原型策略：

1. 抛弃型原型 (Throwaway Prototype) ——用完就扔，只是为了搞清需求
2. 演化型原型 (Evolutionary Prototype) ——不停改进，最后变成最终产品

优点：用户参与度高，需求逐渐清晰 缺点：如果管理不好，可能陷入无休止的修改循环

GEEK NOTE

原型模型特别适合那些用户自己也说不清需求的场景。小红决定用原型法来做一个可以点击的App界面（用Figma画几张图），先给潜在用户看看，收集反馈。这比闷头写代码靠谱多了！

螺旋模型 (Spiral Model)

1988年，Barry Boehm提出了螺旋模型。它把瀑布模型和原型模型结合起来，在每一个螺旋周期中加入了一个非常重要的环节：**风险评估**。

[Diagram: 螺旋模型图。从圆心开始向外旋转，每一圈分为四个象限：确定目标、风险分析、开发与验证、计划下一轮。每一圈代表一个迭代，越靠近圆心成本越低、风险越大。]

螺旋模型的核心：每一轮包含四个步骤：

1. 确定目标——本阶段要做什么
2. 风险分析——评估可能出问题的地方
3. 开发与验证——开发原型或产品
4. 计划下一轮——评审并规划下一步

优点：内置风险分析，适合大型高风险项目 缺点：风险分析需要专家，成本较高；过于复杂，不适合小项目

喷泉模型（Fountain Model）

面向对象方法需要一种新的过程模型——喷泉模型。

为什么叫喷泉？因为水可以**向上喷再落下**——这代表面向对象开发中"各阶段可以**迭代和重叠**"：



特点：

- 迭代——反复进行
- 无缝——各阶段之间没有明显的界限
- 面向对象范型——所有阶段都使用面向对象的概念

适用场景：面向对象开发项目

基于组件的开发模型（Component-Based Development, CBD）

把开发软件变成"搭乐高"——使用现成的标准组件来组装系统。

这要求有一个丰富的**可复用组件库**。

过程：

1. 需求分析
2. 组件分析——找找现成的组件有哪些
3. 架构设计——用组件搭架构
4. 组件定制——适配
5. 组件组装——集成
6. 测试

优点：复用程度高，开发快，质量好 **缺点：**可能找不到完全合适的组件，组件之间的兼容性是问题

SHARPEN YOUR PENCIL

请将下列场景与最合适的过程模型配对：

场景	模型选择
A. 航天飞机控制系统，需求完全确定，不能出错	1. 增量模型

场景	模型选择
B. 一个部门管理小程序，先上线核心功能再扩展	2. 瀑布模型
C. 用户对界面要求很高但描述不清楚	3. 螺旋模型
D. 一个大型金融系统，技术风险很高	4. 原型模型

配对：A-____ / B-____ / C-____ / D-____

（答案见本章末尾）

1.5 统一过程（RUP）

Rational Unified Process（RUP）是Rational公司（后来被IBM收购）开发的重量级软件开发过程框架。

RUP有两个维度：

- **时间维**（Phase）——按时间划分阶段
- **内容维**（Discipline/Workflow）——按工作内容划分

时间维：四个阶段

阶段	英文	目标	里程碑
初始阶段	Inception	确定项目范围	生命周期目标里程碑
细化阶段	Elaboration	建立架构基线	生命周期架构里程碑
构建阶段	Construction	完成系统开发	初始可操作能力里程碑
移交阶段	Transition	交付给用户	产品发布里程碑

内容维：九个核心 workflow

- 六个**核心过程 workflow**：业务建模、需求、分析与设计、实现、测试、部署
- 三个**核心支持 workflow**：配置与变更管理、项目管理、环境

RUP的六大最佳实践：

1. 迭代式开发
2. 管理需求
3. 使用基于组件的架构
4. 可视化建模（UML）
5. 持续验证质量
6. 控制变更

RELAX

RUP在考试中主要考四个阶段的名字和作用，以及它的核心特点：**用例驱动、架构为中心、迭代和增量**。不用深入研究RUP的全部细节，那是架构师的事。

1.6 敏捷开发：反叛军的方法

2001年2月，17个"软件界的老炮儿"在美国犹他州的雪鸟滑雪场聚在一起。他们在滑雪之余（大概），写出了一份改变世界的宣言。

敏捷宣言

个体和互动 高于 流程和工具 **工作的软件** 高于 详尽的文档 **客户合作** 高于 合同谈判 **响应变化** 高于 遵循计划

请注意：不是说右边的东西不重要，而是**左边的更重要**。

敏捷的12条原则

考试中最常出现的几条：

1. 最高优先级是通过尽早和持续交付有价值的软件来满足客户
2. 欢迎变化的需求，即使是在开发后期
3. 经常交付可工作的软件（几周几个月）
4. 业务人员和开发人员每天一起工作
5. 面对面交谈是最有效的沟通方式
6. 团队定期反思如何变得更高效

Scrum：冲刺、站会和看板

Scrum是使用最广泛的敏捷框架。它的核心概念：

三个角色：

- **产品负责人 (Product Owner)** ——管需求，决定做什么
- **Scrum Master** ——管过程，帮团队清除障碍（不是项目经理！）
- **开发团队 (Development Team)** ——干活的人，自组织，跨职能

三个工件：

- **产品待办列表 (Product Backlog)** ——所有要做的事情的总清单
- **冲刺待办列表 (Sprint Backlog)** ——本次Sprint要做的事情
- **增量 (Increment)** ——本次Sprint产出的可交付物

四个事件：

- **冲刺 (Sprint)** ——一个时间盒（通常2-4周），在此期间完成一组任务
- **每日站会 (Daily Scrum)** ——每天15分钟，站着开，回答三个问题：昨天做了什么？今天要做什么？遇到了什么阻碍？
- **冲刺评审 (Sprint Review)** ——Sprint结束时，向利益相关者演示成果
- **冲刺回顾 (Sprint Retrospective)** ——团队自己反思，怎么改进

一个Sprint周期：

———— Sprint (2-4周) —————



[Diagram: 顺时针循环图。四个节点：Sprint Planning → Daily Scrum循环 → Sprint Review → Sprint Retrospective → 回到Planning。]

小红说："Scrum听起来很棒！我们每天站着开15分钟会，谁也别想拖堂！"

极限编程（XP）：把好方法推到极致

如果Scrum是"方法论"，XP就是"工程实践的集大成者"。

XP的核心实践：

1. **结对编程（Pair Programming）** ——两个人用一台电脑写代码
2. **测试驱动开发（TDD）** ——先写测试，再写能通过测试的代码，最后重构
3. **持续集成（Continuous Integration）** ——代码频繁合并到主干
4. **小版本发布**——频繁发布小版本
5. **现场客户**——客户和开发团队坐在一起
6. **代码集体所有制**——代码是大家的，谁都可以改
7. **编码规范**——统一代码风格
8. **重构（Refactoring）** ——持续改进代码结构

TDD的循环：Red → Green → Refactor

Red → 写一个失败的测试（因为它要测的功能还没写）
Green → 写最少的代码让测试通过
Refactor → 优化代码结构，确保测试仍然通过

BRAIN POWER

结对编程：两个人用一台电脑，一个人写代码，一个人在旁边观察和思考。有人觉得这很浪费——两个人干一个人的活。但有人说，结对编程可以减少Bug、共享知识、提升代码质量。

你觉得在什么场景下结对编程最有价值？什么场景下可能不太合适？

1.7 软件生命周期

无论用什么模型，软件的一生大致都要经历这些阶段：

问题定义 → 可行性研究 → 需求分析 → 总体设计 → 详细设计

↓

软件维护 ← 综合测试 ← 单元测试 ← 编码

每个阶段的任务：

阶段	核心问题	产出物
问题定义	要解决什么问题？	问题定义报告
可行性研究	值不值得做？能不能做？	可行性报告
需求分析	系统要做什么？	需求规格说明书（SRS）
总体设计	系统怎么组织？	概要设计文档
详细设计	每个模块怎么实现？	详细设计文档
编码与单元测试	写出能跑的代码	源程序 + 单元测试报告
综合测试	系统整体能用吗？	测试报告
软件维护	运行中修改和完善	维护记录

GEEK NOTE

注意"总体设计"和"详细设计"的区别——这是考试爱考的区分点。总体设计（概要设计）关注"模块之间怎么分工协作"，而详细设计关注"每个模块内部怎么实现"。就像盖房子：总体设计是画户型图（卫生间在哪、卧室在哪），详细设计是画水电管道怎么走。

问与答（There Are No Dumb Questions）

Q：这么多过程模型，我应该选哪个？ A：这就像问"买什么车好"——取决于你的需求。需求稳定选瀑布，需求不清选原型，风险高选螺旋，需要快速迭代选敏捷。小红最后选了Scrum + XP的混合方案。

Q：敏捷开发是不是不需要文档？ A：错！敏捷宣言说"工作的软件高于详尽的文档"，不是"不需要文档"。文档够用就好，别过度。

Q：瀑布模型真的没人用了吗？ A：还在用！航天、医疗、核电站这类"出了Bug会死人"的行业还在用瀑布模型。因为在这些场景下，风险比效率更重要。

Q：软件危机现在解决了吗？ A：没有完全解决，还在"慢性病"阶段。但我们有了更好的工具、方法和管理理念，比1968年好太多了。

BULLET POINTS

- 软件 = 程序 + 数据 + 文档。不是只有代码！
- 软件危机五大表现：预算超支、进度延误、质量低下、维护困难、需求不清
- 软件工程的三个基本要素：方法、工具、过程
- 软件工程的四大目标：正确性、易用性、经济性、可维护性

- 软件工程的四大原则：严格性、关注点分离、模块化、抽象
- 结构化方法：数据与操作分离，自顶向下。工具：DFD、DD、SC图
- 面向对象方法：数据与操作封装。工具：UML各种图
- 瀑布模型：顺序执行，不可逆转。适合需求稳定的项目
- 增量模型：核心功能先行，逐步叠加
- 原型模型：快速构建原型，帮用户搞清楚需求。分抛弃型和演化型
- 螺旋模型：加入了风险评估，每一圈四个步骤。适合高风险大项目
- 喷泉模型：面向对象，迭代、无缝、无边界
- RUP四个阶段：初始（Inception）→ 细化（Elaboration）→ 构建（Construction）→ 移交（Transition）
- 敏捷宣言四个核心价值：个体互动 > 流程工具，工作软件 > 详尽文档，客户合作 > 合同谈判，响应变化 > 遵循计划
- Scrum三个角色：Product Owner、Scrum Master、开发团队。四个事件：Sprint、每日站会、评审、回顾
- XP核心实践：结对编程、TDD（Red-Green-Refactor）、持续集成
- 软件生命周期的8个标准阶段，从问题定义到维护

SHARPEN YOUR PENCIL 答案：

1. 进度延误 2. 维护困难 3. 预算超支 4. 需求不清 5. 质量低下

模型配对答案：A-2（瀑布模型）、B-1（增量模型）、C-4（原型模型）、D-3（螺旋模型）

第2章 软件需求分析：用户说的和他真正想要的，是两回事

小红用原型法做了几个App页面，拿给几个潜在用户看。

"哇，这个页面好漂亮！"第一个用户说。

"但是.....我一般开车的时候不用手机看App，我希望能语音操作。"第二个用户说。

"还有，你们这个只显示空车位数量？我最需要的是**导航到那个空车位**！"第三个用户拍着桌子说。

小红的笔记本上画满了修改。她突然意识到一个问题：

用户嘴上说的"我想要这个"，和他真正需要的"我要解决这个问题"，中间隔着一个太平洋。

2.1 需求分析：做正确的事比正确做事更重要

在软件工程中，有一句话刻在每个程序员的墓碑上：

做错的需求比做错的代码贵100倍。

为什么？因为改Bug只要改代码，改需求可能要推翻整个架构。

需求分析的**任务**就是回答一个问题：**系统到底要做什么？**

四个基本任务：

1. **需求获取 (Elicitation)** ——从用户那里"挖"出需求
2. **需求分析 (Analysis)** ——理解、整理、建模
3. **需求规格化 (Specification)** ——写成正式的文档 (SRS)
4. **需求验证 (Validation)** ——确认写下来的确实是用户要的

需求分析的**原则**（考试爱考）：

1. **信息域原则**——搞清楚数据怎么流、数据是什么、数据结构长什么样
2. **功能分解原则**——把大功能拆成小功能
3. **逻辑建模与物理建模分离原则**——先考虑"做什么"（逻辑），再考虑"怎么做"（物理）

[Diagram: 三层漏斗。最上面一层"用户说的"，中间"分析师理解的"，最下面"实际需求"。三层的面积越来越小，表示信息在传递过程中丢失。旁边写着："每一层都有信息损耗——所以要做需求验证！"]

需求的三大类型

类型	说明	例子
功能需求	系统应该做什么	"App应能显示附近空车位"
非功能需求	系统应该是什么样的	"App应在2秒内返回搜索结果"
约束	开发过程中必须遵守的限制	"必须使用AWS云服务"

WATCH IT!

非功能需求经常被忽视，但它往往比功能需求更致命。用户说"我要搜车位功能"（功能需求），你实现了。但一搜要10秒才出结果（非功能需求未满足），用户直接卸载。——"太慢了"的杀伤力远超"没这个功能"。

2.2 需求获取：怎么让用户说出"心里话"

用户不会把需求写在纸上递给你。你必须主动去"挖"。

方法一：访谈 (Interview)

面对面和用户聊。关键是：**别问"你要什么"，问他"你平时怎么做"**。

有效提问模板：

- "你每天停车的时候是怎么操作的？"
- "最让你烦的是什么？"
- "如果有一个App能.....你觉得有用吗？"

方法二：观察法 (Observation)

用户说他要什么不重要，**看他实际怎么做才重要**。

小红在停车场蹲了三天，观察到：

- 90%的车主在停车场入口减速看标识牌（说明需要"入口引导"功能）
- 30%的车主会绕着停车场转两圈以上（说明需要"精准导航到空位"）

用户从没提过这些需求，但数据告诉你——这就是需求！

方法三：问卷调查（Questionnaire）

适合大规模收集需求。注意：问题要清晰，避免引导性问题。

方法四：原型法（Prototyping）

我们已经讲过了——给用户一个可以看、可以摸的东西。

方法五：头脑风暴（Brainstorming）

把利益相关者聚在一起，畅所欲言。核心规则：**不批评、不评价、每个想法都值得记录。**

方法六：用例分析（Use Case Analysis）

这个方法如此重要，我们专门给它一个章节。见2.3节。

2.3 需求分析与建模：把模糊变清晰

现在你手里有一堆用户访谈记录、观察笔记、问卷数据。然后呢？

你需要**建模**。建模就是用图形化的方式把需求表达出来，让所有人都能看懂。

2.3.1 结构化分析方法

结构化分析方法由三件套组成：

数据流图（Data Flow Diagram, DFD）

DFD描述的是**数据怎么在系统中流动**。它不关心代码怎么写，只关心数据从哪里来、经过什么处理、到哪里去。

四种基本符号：

外部实体：用方框/矩形表示，如"车主"
数据流：用有向箭头表示，如 → 搜索请求 →
处理/加工：用圆圈或圆角矩形表示，如"查询空车位"
数据存储：用两条平行线或开口矩形表示，如"车位数据库"

[Diagram: 小红的停车App的顶层DFD（上下文图）。中间一个大圆圈"停车App系统"，左侧一个方框"车主"，数据流箭头"搜索请求 → 导航结果"双向；右侧一个方框"停车场管理员"，数据流"车位状态更新"双向；底部一个数据存储图标"车位数据库"。]

DFD的分层：一张图画不下？那就分层！

- **顶层图（Context Diagram）**——一个处理（系统本身）+ 所有的外部实体
- **0层图**——把顶层那个大处理拆成几个主要子处理（如：查询车位、预约车位、支付）
- **1层图及以下**——把每个子处理再拆，直到逻辑足够清晰

WATCH IT!

DFD中绝对不能出现"控制流"——比如"如果.....则....."这样的选择逻辑。DFD只画数据流，不画控制流。这是考试常见的陷阱！

数据字典（Data Dictionary, DD）

数据字典是数据流图中每一个数据元素的"词典"。

例如：

数据元素：车位编号
别名：ParkingSpaceID
类型：字符串
长度：10位
取值：前2位为区域代码 + 后8位为数字
说明：唯一标识一个停车位

加工逻辑描述工具

每个"处理"圆圈到底做什么？用以下工具描述：

- 1. **结构化语言**——介于自然语言和编程语言之间的描述方式。比如：

```
IF 用户是VIP会员 AND 车位在500米内
THEN 优先推荐该车位
ELSE 按距离排序推荐
ENDIF
```

- 2. **判定表（Decision Table）**——用表格表示复杂的条件组合
- 3. **判定树（Decision Tree）**——用树形结构表示条件和动作

判定表示例（停车计费规则）：

条件/动作	R1	R2	R3	R4
白天(8:00-20:00)	Y	Y	N	N
是否会员	Y	N	Y	N
每小时收费	5元	8元	3元	5元
封顶	20元	30元	10元	15元

判定树示例（同样逻辑）：



实体-关系图（E-R图）

描述数据之间的关系，是数据库设计的基础。

三个基本元素：

- **实体 (Entity)** ——现实世界中的事物，用矩形表示
- **属性 (Attribute)** ——实体的特征，用椭圆表示
- **联系 (Relationship)** ——实体之间的关联，用菱形表示

联系的类型：

- 1:1（一对一）：一个车位只能停一辆车
- 1:N（一对多）：一个管理员管理多个车位
- M:N（多对多）：一个用户可以使用多个车位（不同时间）

[Diagram: E-R图示例。三个矩形："用户"（属性：用户ID、姓名、手机号），"车位"（属性：车位编号、位置、费率），之间一个菱形"预约"，连线标注M和N，即多对多关系。]

SHARPEN YOUR PENCIL

请用判定表表示小红停车App的"空车位推荐规则"：

- 条件1：距离用户500米内有空车位吗？（有=Y，无=N）
- 条件2：距离1公里内有空车位吗？（有=Y，无=N）
- 条件3：用户是VIP会员吗？（是=Y，否=N）
- 条件4：1公里内有室内车位吗？（有=Y，无=N）

动作A：推荐最近车位 动作B：推荐1公里内最近车位 动作C：不推荐（超过1公里不可用） 动作D：优先推荐室内车位

（提示：D可以和A或B同时发生）

（答案见本章末尾）

2.3.2 面向对象分析方法：UML全家桶

UML (Unified Modeling Language) 是面向对象分析的"通用语言"。它不是一个图，而是一套图，就像工具箱里的各种工具。

UML的图分为两类：

静态图（描述结构—"长什么样"）：

- 用例图 (Use Case Diagram) ——系统提供哪些功能？谁用？
- 类图 (Class Diagram) ——系统有哪些类？它们什么关系？
- 对象图 (Object Diagram) ——某一时刻的系统快照
- 组件图 (Component Diagram) ——软件组件及其依赖
- 部署图 (Deployment Diagram) ——硬件节点和软件组件的部署

动态图（描述行为—"怎么动"）：

- 顺序图/时序图 (Sequence Diagram) ——按时间线展示对象交互
- 协作图/通信图 (Collaboration/Communication Diagram) ——按空间关系展示对象交互
- 状态图 (State Diagram) ——一个对象的状态变化
- 活动图 (Activity Diagram) ——业务流程的控制流

GEEK NOTE

考试中UML图的知识主要集中在**用例图**和**类图**。但**顺序图**和**活动图**也偶尔出现。其他图（部署图、组件图等）了解即可。

用例图 (Use Case Diagram)：谁要做什么？

用例图是最"用户友好"的UML图，因为它描述的是"参与者"（用户/外部系统）和"用例"（系统功能）之间的交互。

核心元素：

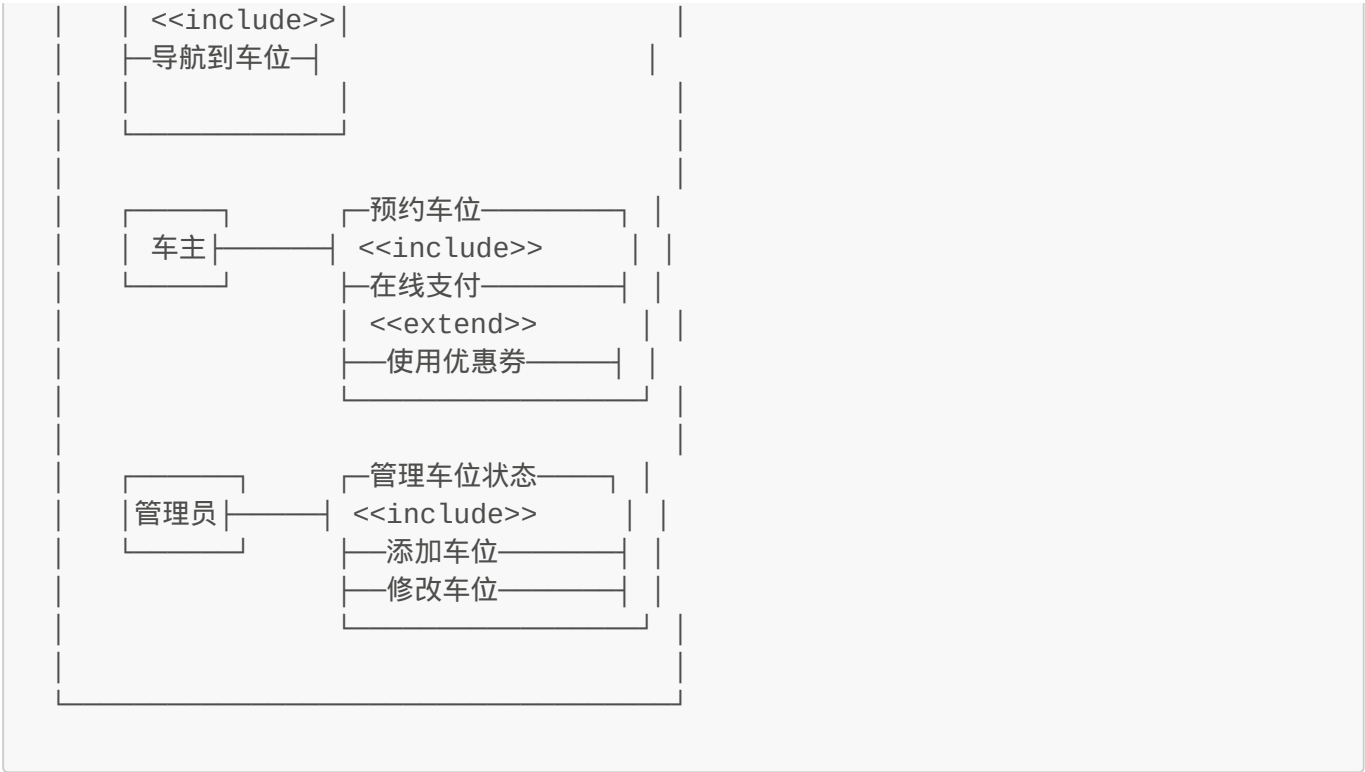
- **参与者 (Actor)** ——用小人图标表示，代表用户或外部系统
- **用例 (Use Case)** ——用椭圆表示，代表系统提供的一项功能
- **系统边界 (System Boundary)** ——用方框把用例围起来

用例图中的四种关系：

1. **关联 (Association)** ——参与者和用例之间的实线，表示"谁用这个功能"
2. **包含 (Include/⟷)** ——一个用例必然包含另一个用例。比如"预约车位"必然包含"验证用户登录"
3. **扩展 (Extend/⟷)** ——一个用例在特定条件下扩展另一个。比如"支付"时可以选择性使用"使用优惠券"
4. **泛化 (Generalization)** ——参与者或用例之间的继承。比如"VIP用户"是"普通用户"的特化

小红的停车App用例图：



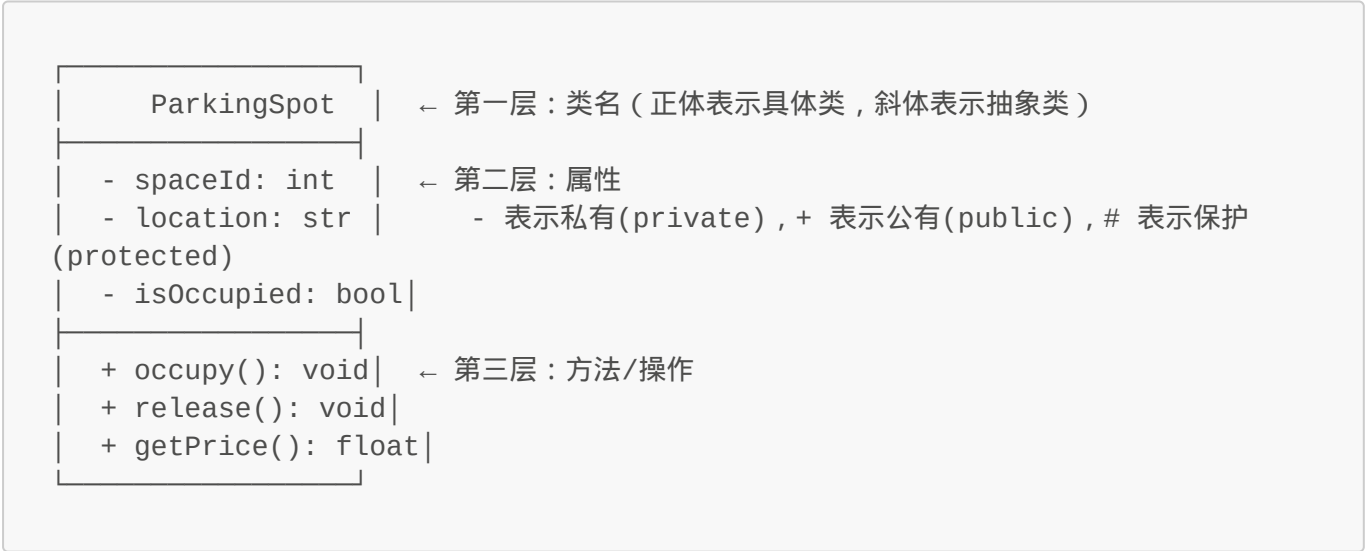


[Diagram: 上述用例图的图形化表示，区分参与者和用例之间的各种关系。]

类图（Class Diagram）：系统里有什么、它们什么关系？

类图是整个UML的核心。它描述系统中的类以及类之间的关系。

一个类在UML中的标准表示（三层矩形框）：



类之间的六种关系（从弱到强）：

关系	UML符号	含义	生活比喻
依赖（Dependency）	虚线+箭头 →	"我在某个方法里用到了你"	我用了一下你的笔

关系	UML符号	含义	生活比喻
关联（Association）	实线	"我知道你，我拥有你的引用"	老师和学生互相知道
聚合（Aggregation）	空心菱形+实线	"你是我的一部分，但你能独立存在"	球队和球员——球员离队了还是球员
组合（Composition）	实心菱形+实线	"你是我的一部分，你离不开我"	订单和订单项——订单删了订单项也没了
泛化（Generalization）	空心三角+实线	"我继承你"	汽车和轿车（轿车是一种汽车）
实现（Realization）	空心三角+虚线	"我实现你定义的接口"	类实现了接口

[Diagram: 小红的停车App完整类图。核心类：User（用户）、ParkingSpace（车位）——抽象父类、IndoorSpace（室内车位）、OutdoorSpace（室外车位）、Reservation（预约记录）、Payment（支付记录）。类之间的关系标注清楚。]

WATCH IT!

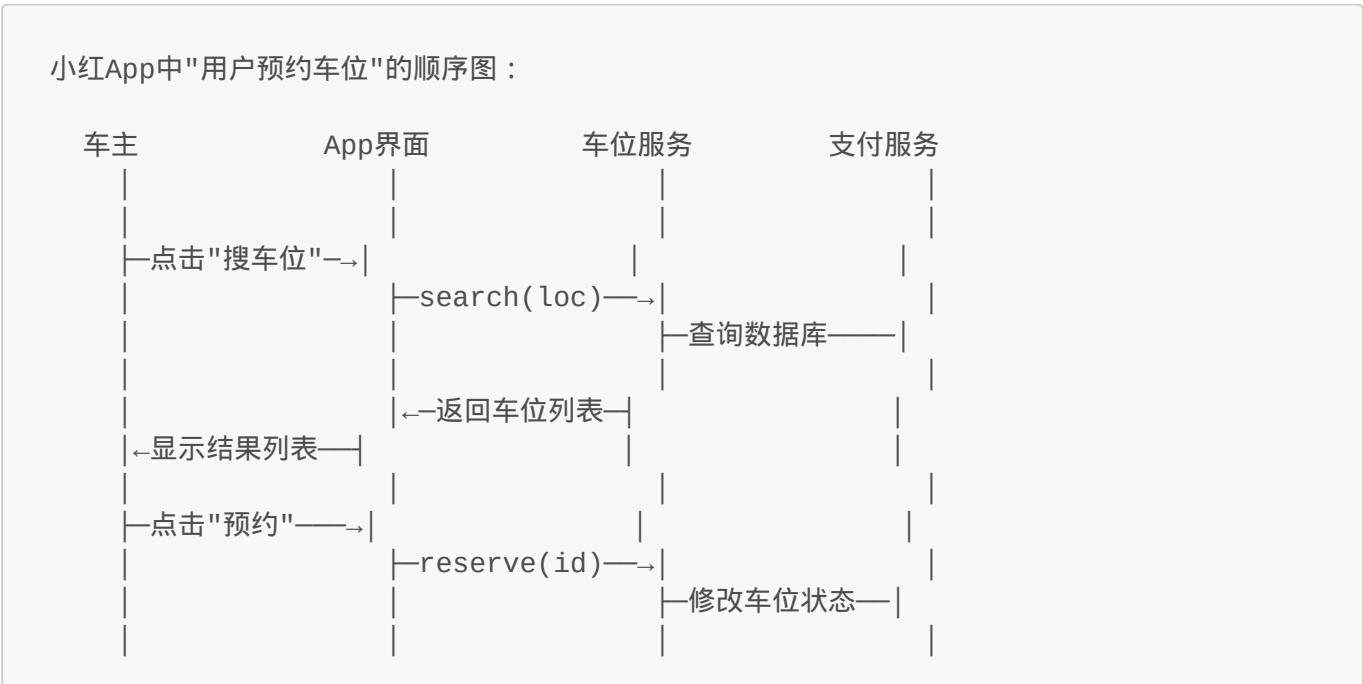
聚合 vs 组合——绝对是考试的高频考点！

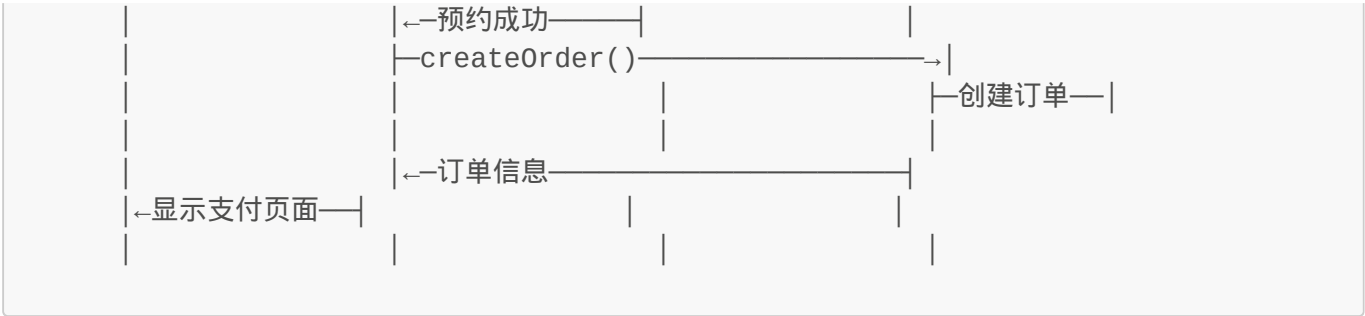
- **聚合**（空心菱形）：部分可以脱离整体独立存在。口诀："分手了还能活"。
- **组合**（实心菱形）：部分的生命周期依赖于整体。口诀："同归于尽"。

举个例子：汽车和发动机是**组合**（发动机是汽车的一部分，汽车报废了发动机也没意义了），停车场和停车位也是**组合**（车位离开停车场就没有意义了）。但停车场和管理员是**关联**。

顺序图（Sequence Diagram）：时间线上的对象对话

顺序图描述了特定场景下对象之间按时间顺序的交互。





顺序图的关键元素：

- **生命线 (Lifeline)** ——从每个对象顶部向下的虚线，表示对象的生命期
- **激活条 (Activation Bar)** ——生命线上的窄矩形，表示对象正在执行操作
- **消息 (Message)** ——带箭头的横线，从左到右表示调用
 - 同步消息：实心三角箭头 → 发送方等待回复（最常用）
 - 返回消息：虚线箭头 --> 表示返回
 - 异步消息：开放箭头 ->> 发送方不等待

协作图 / 通信图 (Communication Diagram)

跟顺序图表达同样的信息但强调对象之间的**连接关系**而不是时间顺序。消息带有序号。



活动图 (Activity Diagram)：业务流程可视化

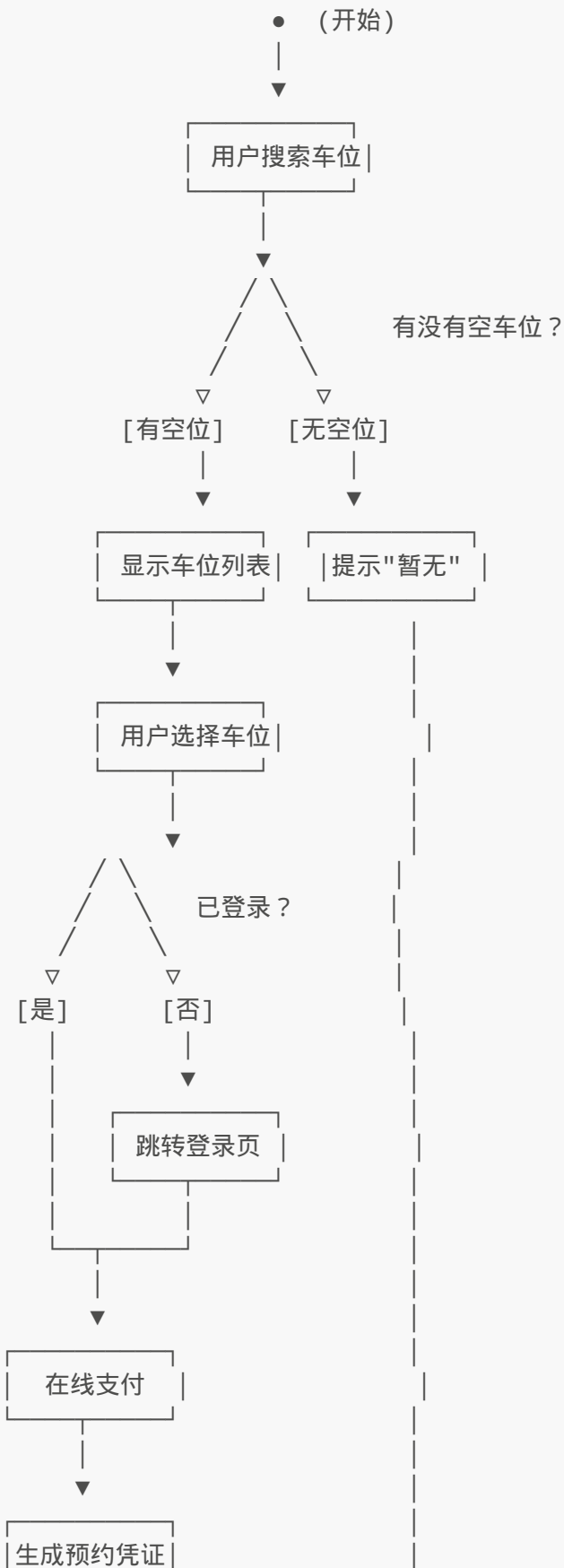
活动图特别适合描述业务逻辑和控制流。

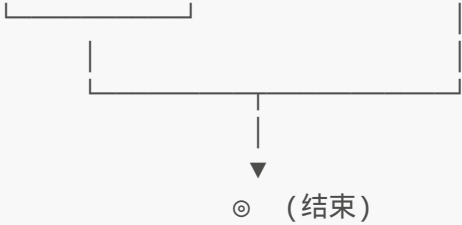
关键元素：

- 圆角矩形：活动/动作节点
- 实心黑点：起始节点

- 实心圆+外圈：终止节点
- 菱形：分支（判断）和合并
- 粗横线：分叉（Fork，并行）和汇合（Join，同步）

小红App"车位预约流程"活动图：



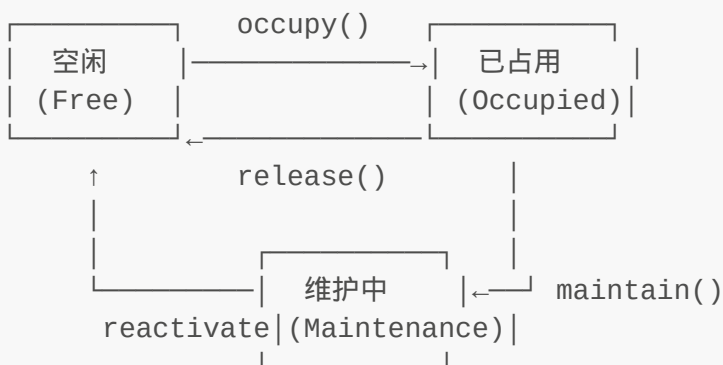


⊙ (结束)

状态图 (State Diagram)：一个对象的一生

描述一个对象在整个生命周期中的状态变化。

车位对象的状态图：



[Diagram: 状态图示例。初始状态 → 空闲状态，空闲 ⇌ 已占用（通过occupy/release转换），已占用或空闲 → 维护中，维护中 → 空闲。]

SHARPEN YOUR PENCIL

阅读下面的用例描述，请画出对应的用例图：

"一个在线停车App中，车主可以搜索空车位、预约车位（预约前必须登录）、在线支付。支付时可以选择使用优惠券。停车场管理员可以管理车位（包括添加、修改、删除车位），其中添加和修改时需要验证管理员权限。"

请画出完整的用例图，标注所有关系（include、extend、泛化）。

（答案见本章末尾）

BE THE ANALYST：小红的停车App需求分析

你是小红团队的需求分析师。请根据以下信息完成需求分析的三个任务：

背景：某商圈三个停车场（A、B、C）希望联合推出智能停车服务。车主可以通过一个App实时查看三个停车场的空余车位，预约并导航到指定车位，在线支付停车费。

任务1：识别至少3个功能需求、2个非功能需求和1个约束。**任务2**：画一张顶层数据流图（上下文图），标注外部实体和数据流。**任务3**：用结构化语言描述"搜索空车位"的处理逻辑。

（参考答案见本章末尾）

2.4 需求规格说明书（SRS）

花了这么大力气获取需求、分析需求，最后要交付什么？

软件需求规格说明书（Software Requirements Specification, SRS）

SRS是需求分析的**最终产出物**，是开发团队和用户之间的"合同"。IEEE 830标准定义了SRS应该包含的内容。

SRS的典型内容结构：

1. 引言（目的、范围、定义、参考文献）
2. 总体描述（产品视角、用户特征、假设与依赖）
3. 具体需求（功能需求、非功能需求、接口需求）
4. 附录与索引

好的SRS有什么特征？（考试爱考）：

- **正确性**——每个需求确实是用户想要的
- **无歧义性**——每条需求只有一种解释
- **完整性**——所有重要的需求都包含了
- **一致性**——需求之间没有矛盾
- **可验证性**——每条需求都能通过测试验证
- **可追踪性**——每个需求的来源和去向都清楚
- **可修改性**——修改一个需求不影响其他需求

需求验证的技术：

- **需求评审**——同行专家一条一条审查
- **原型验证**——让用户操作原型确认
- **测试用例设计**——根据需求设计测试，看看需求能否被测
- **场景走查**——模拟用户的真实使用场景走一遍

RELAX

SRS的七个特征考试会考。口诀："正无完一可修可"——正确性、无歧义性、完整性、一致性、可验证性、可追踪性、可修改性。这七个词里选四个让你辨认是高频题型。

问与答（There Are No Dumb Questions）

Q：DFD和UML的类图有什么区别？ A：DFD关注的是"数据怎么流"（功能视角），类图关注的是"有哪些类及它们的关系"（结构视角）。DFD属于结构化方法，类图属于面向对象方法。

Q：顺序图和通信图是不是画一个就够了？ A：是的！它们表达的是同样的信息，只是视角不同。UML把它们称为"同构的"（Isomorphic）。考试中顺序图出现得更多。

Q：包含《include》和扩展《extend》怎么区分？ A：include是"必须的"——比如"预约"必然包含"登录"。extend是"可选的"——比如"支付"时可选"使用优惠券"。口诀："include是必须，extend是可选。"

Q：需求分析和需求获取是一回事吗？ A：不是。获取是"挖矿"（从用户那里把需求挖出来），分析是"冶炼"（把挖出来的原始需求整理成结构化的模型）。先获取，后分析。

BULLET POINTS

- 需求分析四大任务：获取→分析→规格化→验证
- 需求分析三大原则：信息域原则、功能分解原则、逻辑与物理分离原则
- 需求的三大类型：功能需求、非功能需求、约束
- 需求获取六种方法：访谈、观察、问卷、原型、头脑风暴、用例分析
- DFD四种符号：外部实体、数据流、处理、数据存储。记住：DFD只画数据不画控制
- 数据字典（DD）是DFD中每个数据元素的定义词典
- 判定表适合复杂条件组合，判定树更直观易懂
- E-R图三要素：实体（矩形）、属性（椭圆）、联系（菱形）。联系类型：1:1、1:N、M:N
- UML九种图分两类：静态图5种（用例、类、对象、组件、部署），动态图4种（顺序、协作、状态、活动）
- 用例图四关系：关联、包含（必须）、扩展（可选）、泛化（继承）
- 类图六关系（强到弱）：实现、泛化、组合、聚合、关联、依赖
- 聚合"分手了还能活"，组合"同归于尽"
- 顺序图强调时间顺序（上下方向），通信图强调空间关系
- 活动图=高级流程图，状态图=单个对象的状态机
- SRS是需求分析的最终产出物，是开发与用户之间的"合同"
- SRS七大特征：正确、无歧义、完整、一致、可验证、可追踪、可修改

SHARPEN YOUR PENCIL 答案（判定表）：

条件/动作	R1	R2	R3	R4	R5
500米内有空车位	Y	Y	N	N	N
1公里内有空车位	—	—	Y	Y	N
是VIP会员	N	Y	N	Y	—
1公里内有室内车位	—	—	—	Y	—
推荐最近车位	X	X			
优先推荐室内车位				X	
推荐1公里内最近			X		
不推荐					X

SHARPEN YOUR PENCIL 答案（用例图）：

关键点：车主关联"搜索空车位"、"预约车位"、"在线支付"。预约车位包含《include》登录。在线支付扩展《extend》使用优惠券。管理员关联"管理车位"（泛化到添加、修改、删除）。添加和修改包含验证权限。

BE THE ANALYST 参考答案：

任务1：功能需求：①查看实时空车位 ②预约指定车位 ③导航到车位 ④在线支付（至少3个）。非功能需求：①响应时间<2秒 ②支持同时5000人在线。约束：必须兼容iOS和Android。

任务2：顶层DFD——中心"智能停车系统"，外部实体"车主"（搜索请求输入，车位信息输出）、"停车场管理员"（车位状态输入）、"支付网关"（支付请求/确认）。

任务3：结构化语言——

```
接收用户输入的搜索条件（位置、距离范围）
查询车位数据库中状态为"空闲"的车位
按距离排序
IF 结果为空 THEN 提示"附近无空车位"
ELSE 返回前20条结果
ENDIF
```

第3章 软件设计：把"要做什么"变成"怎么做"的蓝图

小红团队的需求文档终于写完了。她拿着厚厚一叠纸（其实就一份SRS加上各种UML图），感觉终于搞清楚了自己要做什么。

"现在可以开始写代码了吧？"小刚摩拳擦掌。

"等一下！"小红按住他的手，"你知道盖房子之前要先画图纸吗？你知道做菜之前要先切菜配料吗？写代码之前，要先**设计**。"

"不然你写到一半发现数据结构不对、接口设计不合理、模块之间耦合得解不开——那就只能删库重来了。"

小刚倒吸一口凉气："那.....那我们怎么设计？"

3.1 软件设计基础：从"做什么"到"怎么做"

如果说需求分析回答的是 "**做什么**"（**What**），那么软件设计回答的是 "**怎么做**"（**How**）。

软件设计分两层：

```
需求分析（逻辑模型：系统要做什么）
    |
    ▼
概要设计 / 总体设计（系统架构：模块怎么分工）
    |
    ▼
详细设计（模块内部：每个模块怎么实现）
```

概要设计 vs 详细设计

维度	概要设计（总体设计）	详细设计
视角	宏观，全局	微观，局部
关注点	模块之间的"接口"和"关系"	每个模块内部的"算法"和"数据结构"
产物	模块结构图（SC图）、系统架构图	流程图、N-S图、伪代码
比喻	设计一栋楼——哪个房间在哪层	设计每个房间——电线水管怎么走
回答的问题	"系统分成哪几块？"	"每块具体怎么实现？"

软件设计的基本原则（四大金刚）

1. 抽象（Abstraction）

忽略细节，抓住本质。你要设计一个"支付系统"，第一步不是想用支付宝还是微信，而是先抽象成"发起支付 → 处理支付 → 返回结果"这个过程。

三种抽象层次：

- **过程抽象**——一个命名的操作序列（比如"搜索空车位"这个功能）
- **数据抽象**——一个命名的数据集合（比如"车位"这个数据结构）
- **控制抽象**——一个命名的控制机制（比如"事务管理"）

2. 逐步求精（Stepwise Refinement）

自顶向下，从高层抽象逐步细化到可执行的代码。

第1层：搜索空车位

|

第2层：获取用户位置 → 查询附近车位 → 按距离排序 → 返回结果

|

第3层：获取用户位置 → GPS定位 → 获取经纬度

查询附近车位 → `SELECT * FROM parking_spots WHERE status='free' AND distance < 1000`

按距离排序 → `ORDER BY distance ASC`

返回结果 → 封装成JSON → 返回给前端

3. 模块化与信息隐藏

把系统拆成独立的模块，每个模块只暴露**需要暴露的接口**，隐藏内部实现细节。

就像汽车的油门——你不需要知道踩下去以后燃油是怎么喷射的，你只需要知道"踩油门=加速"这个接口。

信息隐藏的好处：

- 模块内部修改不影响外部（比如把数据库从MySQL换成PostgreSQL，只要接口不变）
- 降低系统复杂度

- 便于测试和维护

4. 模块独立性

这是软件设计的**核心目标**。衡量标准就是两个词：**耦合和内聚**。

耦合与内聚：软件设计的"黄金法则"

高内聚、低耦合——这六个字是所有软件设计的终极追求。

耦合（Coupling）：模块之间的关联程度

你希望耦合越**低**越好。就像邻居——最好各过各的，不要你家马桶堵了邻居家也跟着臭。

耦合类型	程度	说明	示例
无直接耦合	最理想	模块之间没有直接关系	支付模块和日志模块
数据耦合	低	只通过简单数据（参数）通信	search(location)返回车位列表
标记耦合	中低	通过数据结构（如对象）通信	search(userProfile)传整个用户对象
控制耦合	中	传递控制信息（如flag）	process(type="reserve")——type控制行为
外部耦合	中高	共享同一个外部设备或全局数据	多个模块读同一个全局配置文件
公共耦合	高	共享同一个全局数据区	多个模块读写同一个全局变量
内容耦合	最糟糕	一个模块直接访问另一个模块内部	模块A直接修改模块B的私有变量

口诀（从好到坏）：无数标记控外公内——无直接耦合、数据耦合、标记耦合、控制耦合、外部耦合、公共耦合、内容耦合。

内聚（Cohesion）：模块内部各成分之间的关联程度

你希望内聚越**高**越好。模块内部应该像一个家庭——一家人同心协力做一件事。

内聚类型	程度	说明	示例
偶然内聚	最低	模块内各部分毫无关系，随机放一起	一个"杂项工具"模块
逻辑内聚	低	逻辑上相似但功能不同	一个模块处理所有类型的"IO操作"
时间内聚	中低	在同一时间段执行的放在一起	系统初始化时一口气做所有init
过程内聚	中	按特定顺序执行（但数据不相关）	先检查权限再记录日志
通信内聚	中高	处理相同的输入/输出数据	对同一份车位数据做查询和统计
顺序内聚	高	前一个的输出是后一个的输入	获取GPS位置 → 查询附近车位 → 导航
功能内聚	最高	模块只做一件事且做好	"预约车位"模块只负责预约逻辑

口诀（从差到好）：偶逻时过通顺功——偶然、逻辑、时间、过程、通信、顺序、功能。

WATCH IT!

耦合和内聚的分类是考试的超级重点！不仅会考"以下哪个耦合度最高/最低"，还会考给你一个具体场景让你判断属于哪种耦合或内聚。比如："模块A通过全局变量与模块B通信"——这是**公共耦合**。"一个模块包含初始化系统、创建数据库连接、打开日志文件三个操作"——这是**时间内聚**（都在初始化时做）。

SHARPEN YOUR PENCIL

判断以下场景的耦合类型和内聚类型：

- 1. "查询车位模块"通过参数传递用户位置给"导航模块"——耦合类型？_____
- 2. 模块A直接goto跳入模块B的代码内部——耦合类型？_____
- 3. 一个模块打开文件、读取文件、解析JSON、写入数据库——内聚类型？_____
- 4. 多个模块共享同一个全局数据结构"车位列"——耦合类型？_____
- 5. "支付模块"只做一件事：处理支付——内聚类型？_____

(答案见本章末尾)

3.2 结构化设计方法

结构化设计是对结构化分析的延续：先从DFD导出系统结构，再用各种工具细化每个模块。

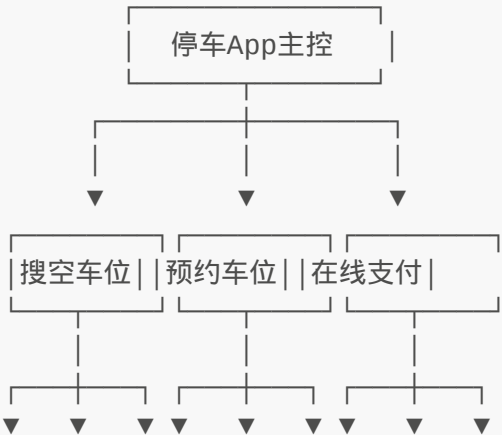
3.2.1 概要设计：从DFD到模块结构图

模块结构图（Structure Chart, SC图）描述模块之间的**调用关系**和**数据传递**。

SC图的基本符号：

- 矩形框：一个模块
- 实线箭头：模块A调用模块B
- 空心小箭头+数据名：传入/传出的数据
- 空心点+数据名：传入/传出的控制信息（flag）
- 小菱形+实线：条件调用
- 弧线+实线：循环调用

SC图示例——小红停车App的部分结构：



GP	DB	显	验	生	发	支	发	通
S	查	示	证	成	送	付	票	知
定	询	结	用	预	通	接	生	用
位		果	户	约	知	口	成	户

3.2.2 面向数据流的设计方法

这是结构化设计的核心策略。方法论是：分析你的DFD是什么样的结构，就用什么方法推导出SC图。

DFD有两种基本结构，对应两种设计策略：

变换分析（Transform Analysis）

当DFD呈现 "输入→处理→输出" 的线性结构时使用。

DFD结构：
外部输入 → 输入流 → 变换中心 → 输出流 → 外部输出

SC图结构：

主控模块

▼

▼

▼

输入控制

变换控制

输出控制

▼

▼

▼

输入模块

变换模块

输出模块

变换分析步骤：

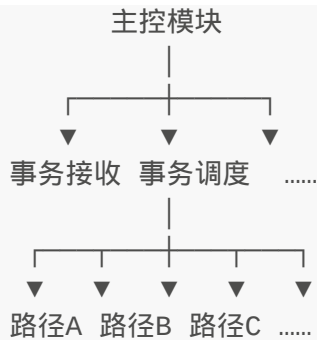
- 1. 在DFD中找出**变换中心**（核心逻辑处理部分）
- 2. 向前追踪找出**传入路径**（输入流）
- 3. 向后追踪找出**传出路径**（输出流）
- 4. 设计顶层模块（主控模块），一级模块（输入控制、变换控制、输出控制）
- 5. 逐步分解下层模块

事务分析（Transaction Analysis）

当DFD的核心是一个 "事务中心"——根据输入的类型分派到不同的处理路径时使用。

DFD结构：
输入 → 事务中心（分析类型） —→ 路径A的处理
 └→ 路径B的处理
 └→ 路径C的处理

SC图结构：



小红停车App的例子：用户操作可能是"搜车位"、"预约"、"支付"、"查看历史"——这就是一个事务中心，根据不同操作分派到不同模块。

GEEK NOTE

考试中变换分析和事务分析是选择题的常客。记住：DFD是线性结构用变换分析，DFD有分叉/路由中心用事务分析。

3.2.3 模块化设计启发式规则

结构化设计有一些"经验法则"：

1. **模块规模适中**——太小（功能太碎）太大（耦合太重）都不好
2. **扇入/扇出合理**：
 - 扇入（被调用的模块数）——越高越好，表示复用性好
 - 扇出（调用的模块数）——一般控制在3-5个
3. **深度、宽度适中**——模块调用层次不要太深（难以理解）也不要太浅（功能挤一起）
4. **作用域在控制域内**——一个模块的判断影响范围（作用域）应该在它所控制的模块范围（控制域）之内

3.2.4 详细设计工具

概要设计确定了模块结构，详细设计要画出每个模块内部的实现细节。

程序流程图（Program Flow Chart）

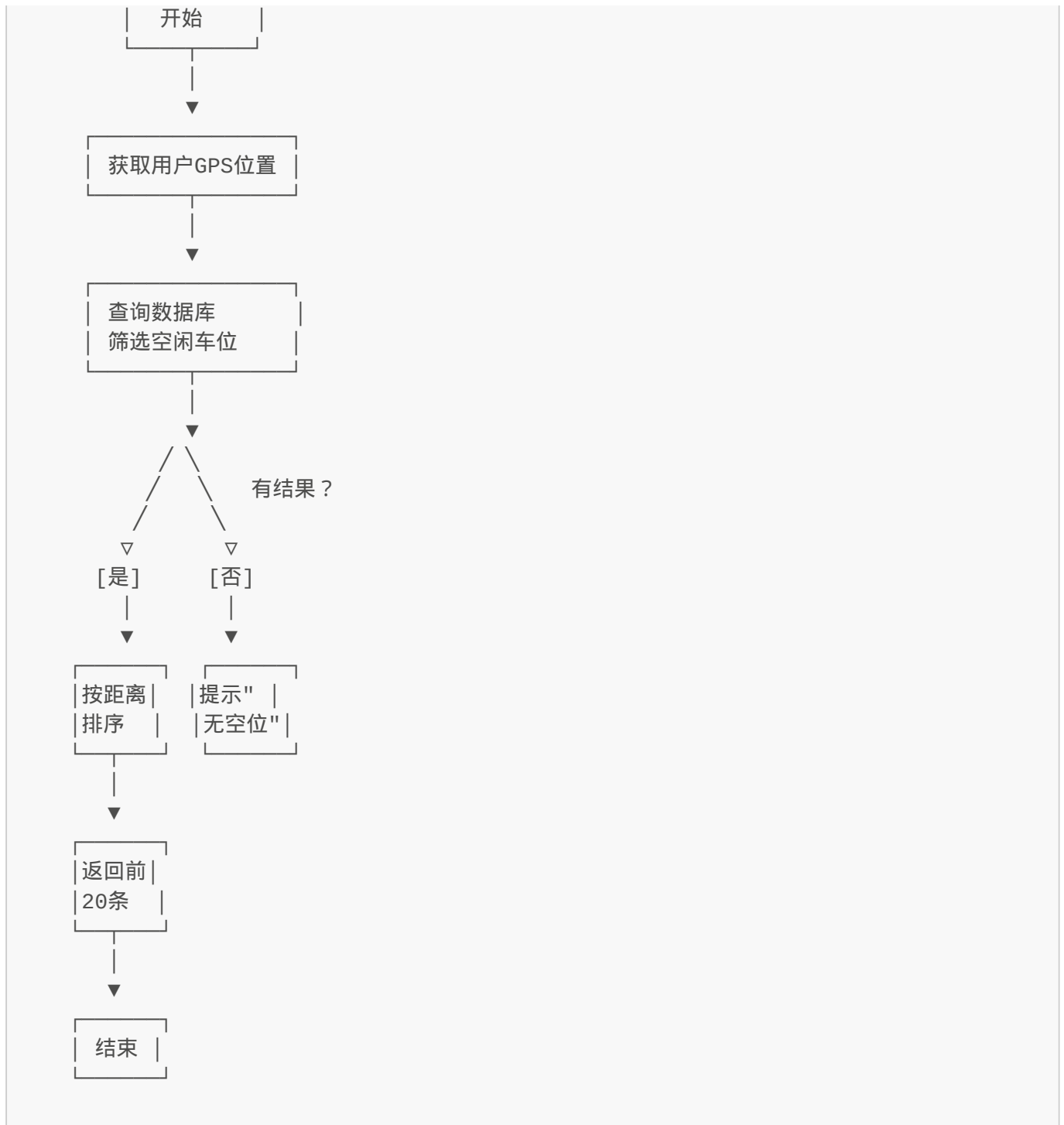
最经典、最直观的详细设计工具。

基本符号：

- 椭圆/圆角矩形：开始/结束
- 矩形：处理步骤
- 菱形：判断/选择
- 箭头：控制流方向
- 平行四边形：输入/输出

流程图示例——搜索空车位模块：



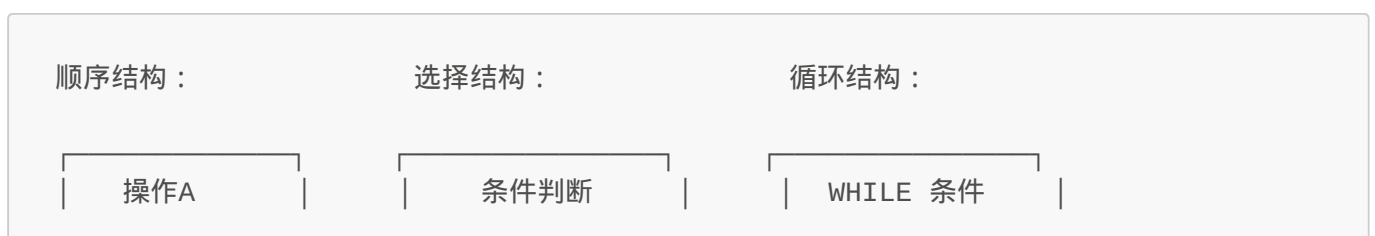


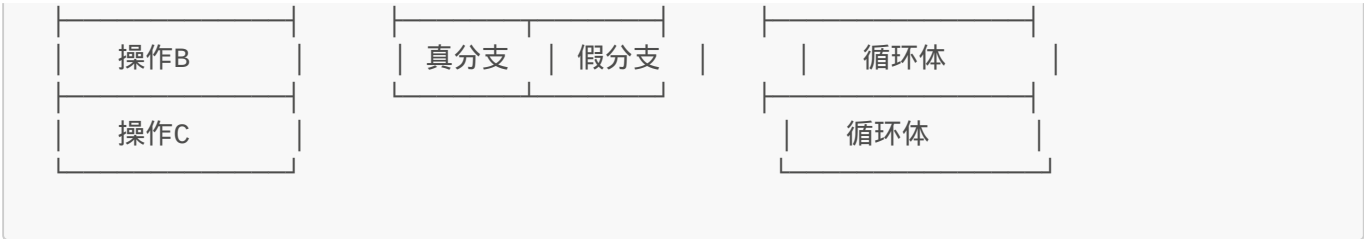
流程图的缺点：可以任意画线（GOTO风格）导致结构不清晰。

盒图（N-S图，Nassi-Shneiderman Diagram）

为了解决流程图"GOTO到处飞"的问题，盒图强制结构化——只有三种基本控制结构，不能画任意跳转线。

N-S图的基本元素全部由嵌套的矩形框构成：





N-S图的优点：结构清晰、没有GOTO、层次分明。缺点：修改不方便。

PAD图（Problem Analysis Diagram，问题分析图）

从日本引进的设计工具，用树形结构表示控制流。

PAD图基本符号：

P1

— 操作/处理节点

C

P1

— 条件选择 (if-then-else)

P2

C

=1=

P1

— 多分支选择 (switch/case)

=2=

P2

WHILE

P

— while循环

PAD图优点：既清晰又容易转换为代码。

判定表与判定树

在需求分析那一章我们已经见过面了。它们在详细设计中同样有用——尤其是描述那些条件组合复杂的逻辑。

过程设计语言（PDL / 伪代码）

用类似编程语言的形式描述模块逻辑，但不拘泥于具体语法。

```
PROCEDURE 搜索空车位(用户位置, 距离范围)
BEGIN
```

获取用户的GPS坐标

```
查询车位数据库: SELECT * FROM spots
                  WHERE status = 'free'
                  AND distance(spot_location, 用户位置) <= 距离范围
                  ORDER BY distance ASC
```

如果(结果为空):

返回 "附近无空车位"

否则:

返回 前20条结果

END

RELAX

详细设计工具考试主要考各种工具的名称和特点。口诀：流程图灵活但不规范，N-S图强制结构化，PAD图树形表达易转代码，判定表/判定树搞定复杂条件，PDL是书面代码。会辨认、会选工具即可。

SHARPEN YOUR PENCIL

请将下列设计任务与最合适的详细设计工具配对：

任务	工具
A. 描述一个包含复杂条件判断的计费逻辑	1. 流程图
B. 描述一个不允许GOTO的高安全性模块	2. N-S图
C. 快速画一个处理流程给领导看	3. PAD图
D. 为后续自动生成代码做准备	4. 判定表

配对：A-___ / B-___ / C-___ / D-___

(答案见本章末尾)

3.3 面向对象设计方法

面向对象设计（OOD）是面向对象分析（OOA）的延续。OOA产出逻辑模型（类图、用例图），OOD把这些模型转化为可实现的设计。

面向对象设计的概念

OOD的核心任务：

1. **精化类模型**——添加更多细节（属性的类型、方法的签名）
2. **设计类的接口**——确定每个类暴露哪些方法
3. **设计交互**——用顺序图/通信图细化对象之间的协作
4. **划分模块/子系统**——把相关类组织成包（Package）或组件
5. **设计持久化**——对象怎么存储到数据库（对象-关系映射）

设计模式简介

1994年，四人帮（GoF）写了一本书《设计模式》，总结了23种经典的面向对象设计模式。

设计模式 = 已命名的、经过验证的、可复用的面向对象设计解决方案。

设计模式分三类：

创建型模式（Creational Patterns）——管"创建对象"的

模式	一句话	小红的场景
单例（Singleton）	全局只有一个实例	停车App的配置管理器，只能有一个
工厂方法（Factory Method）	让子类决定创建哪个对象	创建不同类型的支付方式（微信/支付宝）
抽象工厂（Abstract Factory）	创建一组相关对象	同时创建支付对象和对应的发票对象
建造者（Builder）	分步骤构建复杂对象	构建一个复杂的"预约订单"对象
原型（Prototype）	克隆现有对象	从模板复制一个车位配置

结构型模式（Structural Patterns）——管"组织类和对象"的

模式	一句话	小红的场景
适配器（Adapter）	两个不兼容的接口对接	对接不同的停车场API
装饰器（Decorator）	动态给对象加功能	车位可以动态加"带充电桩"、"带遮阳棚"属性
代理（Proxy）	控制对对象的访问	支付服务的代理（控制访问和日志）
外观（Facade）	提供统一简化接口	停车服务的统一门面
组合（Composite）	树形结构	停车场的区域-楼层-车位层级结构

行为型模式（Behavioral Patterns）——管"对象怎么交互"的

模式	一句话	小红的场景
观察者（Observer）	状态变化自动通知	车位状态变化 → 通知等待的车主
策略（Strategy）	算法族可互换	不同时段的计费策略（白天/夜间/节假日）
命令（Command）	把请求封装成对象	预约、取消预约、改约——都是命令对象
模板方法（Template Method）	定义算法骨架，子类填空	支付流程骨架：验证 → 扣款 → 确认 → 通知
状态（State）	对象行为随状态改变	车位的状态机：空闲/占用/维护

BRAIN POWER

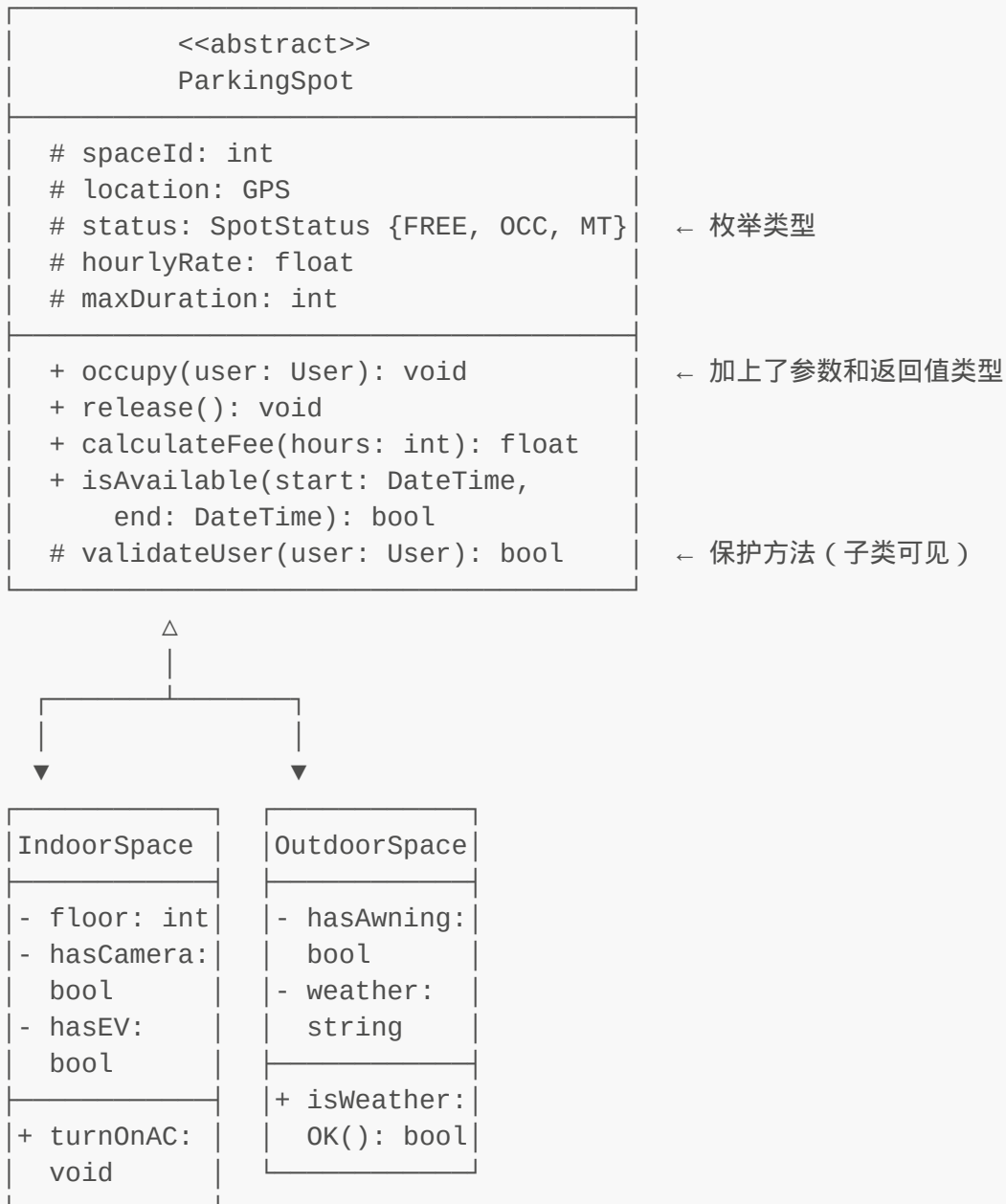
小红App中的计费逻辑有三种：白天计费、夜间计费、节假日计费。如果用策略模式，她可以定义三个计费策略类，在运行时根据时间切换。如果不用设计模式，她可能会写一堆if-else判断，每次新增一种计费规则就得改代码。

你觉得还有哪些设计模式适合小红的停车App？试着举出两个并解释为什么。

UML类图的详细设计

在设计阶段，OOA阶段画的类图要被"精化"——加上更多细节：

精化后的ParkingSpot类图（设计阶段）：



组件图与部署图

这两个图关注的是软件的**物理层面**——代码怎么打包、部署到哪些服务器上。

组件图（Component Diagram）：

- 展示软件组件（如.jar文件、.war文件、可执行文件）及其依赖关系
- 符号：矩形+两个小矩形标记 = 组件。虚线箭头 = 依赖。

部署图 (Deployment Diagram) :

- 展示硬件节点和软件组件的部署关系
- 符号：立方体 = 节点（服务器/设备），内部可以放组件
- 小红的部署图：一个"云服务器"节点运行"停车App后端"，一个"MySQL数据库"节点，一个"Redis缓存"节点

RELAX

组件图和部署图在考试中不是重点。知道它们是什么、长什么样就行。真正考UML的重头戏是**类图**和**用例图**。

3.4 用户界面设计

小红兴奋地说："好了好了，后台设计好了。小美，你画App界面吧！随便画好看点就行。"

小美翻了个白眼："UI设计不是'画好看点'那么简单....."

用户界面设计的三大原则

1. 用户控制原则

- 用户应该是主导者，不是被动使用者
- 提供"撤销"按钮——用户犯错了可以回头
- 提供明显的"退出"和"取消"

2. 减轻记忆负担原则

- 人类短期记忆只有 7 ± 2 个单元（Miller定律）
- 不要强迫用户记住前一页的信息
- 图标+文字比纯图标好
- 使用下拉菜单而不是让用户手动输入一切

3. 一致性原则

- 同样的操作在所有页面上保持一致
- 红色=警示/删除，绿色=确认，蓝色=链接
- 菜单的位置、快捷键、术语全统一

人机交互设计准则（Nielsen十大可用性原则）

Jakob Nielsen在1994年提出——虽然过去30年了，依然是一条一条金科玉律：

1. **系统状态可见性**——用户永远知道"正在发生什么"（比如加载中的进度条）
2. **系统与现实匹配**——用用户的语言，而不是程序员的术语
3. **用户控制和自由**——提供"紧急出口"（撤销/重做）
4. **一致性和标准**——别让用户猜"不同的词是不是同一个意思"
5. **防止错误**——在设计层面就避免用户犯错（比如不可用的按钮变灰）
6. **识别而非回忆**——让用户"认"而不是"想"
7. **灵活性与效率**——新手和老手都要能用（快捷键给老手）
8. **美观与简约**——不相关的东西去掉

9. 帮助用户识别和纠正错误——错误提示告诉用户"为什么错了"和"怎么改"
10. 帮助与文档——最好的系统不需要文档，但万一需要，它得是好用的

GEEK NOTE

考试考UI设计通常是选择题或简答题。"Nielsen十大原则"中最常考的是第5条（防错）和第1条（状态可见）。另外，Miller定律（ 7 ± 2 ）是面试和考试的经典考点。

BE THE ANALYST：设计小红的停车App

你是小红团队的"架构师"。请完成以下设计任务：

任务1：根据之前的需求分析，画出停车App的模块结构图（SC图）。至少要有三层模块。（提示：顶层"停车App主控"，下面至少分出"用户管理"、"车位查询"、"预约管理"、"支付管理"四个一级模块。）

任务2：选择"搜索空车位"这个模块，用N-S图画出它的详细设计。

任务3：分析"预约管理"模块和"支付管理"模块之间的耦合关系——它们应该用什么方式通信？属于哪种耦合类型？如何降低耦合？

（参考答案见本章末尾）

问与答（There Are No Dumb Questions）

Q：概要设计和详细设计到底怎么区分？ A：概要设计画SC图，关注"模块之间"；详细设计画流程图/N-S图，关注"模块内部"。就好像先设计整栋楼的户型（卫生间在哪、卧室在哪），再设计每个房间的水电走向。

Q：耦合是不是越低越好？ A：原则上是的。但注意：有些耦合是不可避免的。比如"预约管理"和"支付管理"之间必然有数据交互。我们的目标是尽量减少**不必要的**耦合，特别是公共耦合和内容耦合，那是绝对要避免的。

Q：所有系统都适合用面向对象设计吗？ A：不一定。简单的批处理程序、纯数学计算程序，用结构化方法（甚至直接用函数）就很好。面向对象在复杂交互系统中优势最大。

Q：设计模式需要背23种吗？ A：不用！考试会让你"简要说明三种设计模式"，只要能写出几个常见的（单例、工厂、观察者、策略）并解释其用途就够了。重点是理解，不是背诵。

BULLET POINTS

- 软件设计分两层：概要设计（模块间关系）+ 详细设计（模块内实现）
- 设计四大原则：抽象、逐步求精、模块化与信息隐藏、模块独立性
- 耦合（越低越好）：从高到低——内容 > 公共 > 外部 > 控制 > 标记 > 数据 > 无直接
- 内聚（越高越好）：从低到高——偶然 < 逻辑 < 时间 < 过程 < 通信 < 顺序 < 功能
- "高内聚、低耦合"是软件设计的最高准则
- 结构化设计：面向数据流，DFD → SC图。变换分析（线性DFD）和事务分析（分叉DFD）
- 详细设计工具：流程图（灵活）、N-S图（强制结构化）、PAD图（树形）、PDL（伪代码）、判定表/树
- 面向对象设计：精化OOA的类模型，设计交互和接口
- 设计模式三大类：创建型（管new）、结构型（管组织）、行为型（管交互）
- 单例=唯一实例，工厂=子类决定创建，观察者=自动通知，策略=算法互换
- UML类图在设计中要精化：加上属性类型、方法签名、可见性（+/-/#）

- 组件图看软件打包，部署图看硬件部署
- UI设计三大原则：用户控制、减轻记忆负担（7±2）、一致性
- Nielsen十大原则：状态可见、防错设计、识别而非回忆.....

SHARPEN YOUR PENCIL 答案（耦合/内聚）：

1. 数据耦合 2. 内容耦合 3. 通信内聚（都是对同一份数据的操作） 4. 公共耦合 5. 功能内聚

SHARPEN YOUR PENCIL 答案（设计工具配对）：A-4（判定表最适合复杂条件组合）、B-2（N-S图强制结构化，无GOTO）、C-1（流程图最直观易懂）、D-3（PAD图容易自动转换为代码）

BE THE ANALYST 参考答案：任务1：SC图——顶层"停车App主控"，一级模块"用户管理"、"车位查询"、"预约管理"、"支付管理"、"通知管理"。每个一级模块下细分二级模块（如"车位查询"下有"GPS定位"、"数据库查询"、"结果排序"、"结果显示"）。

任务2：N-S图——外层是顺序结构（获取位置→查询→判断→返回），内部是条件选择（有结果/无结果）。

任务3：预约管理模块向支付管理模块传递"预约信息（车位ID、时间段、金额）"→这是**数据耦合**（最低的耦合）。如果改成共享一个全局"当前订单对象"就是公共耦合（坏的）。要降低耦合，就应该坚持"传参数，不共享全局状态"。

第4章 软件编码与实现：键盘敲下去的那一刻

设计文档终于完工了。小刚的双手已经在键盘上空颤抖了三天。

"我可以开始写代码了吗？"

小红点点头。小刚的手指像雨点一样落下去——"噼里啪啦噼里啪啦"。

15分钟后.....

"糟了，变量名全用的拼音首字母，我自己都看不懂了....."

小美凑过来："你怎么把所有逻辑写在了一个函数里？这函数有600行了！"

小红叹了口气："编码，也是一门学问啊。"

4.1 编码：不只是"写代码"

编码（Coding）是把详细设计转化成**计算机能理解和执行的程序**的过程。

但编码远不止"翻译"这么简单。好的代码像好的文章——结构清晰、表达准确、其他人能读懂。

4.1.1 程序设计语言的选择

选择编程语言就像选择交通工具——看你从哪出发、要去哪、有多远、载几个人。

影响语言选择的因素：

因素	说明
项目类型	Web应用 → Java/Python/JS；系统软件 → C/C++/Rust；数据分析 → Python/R
团队技能	团队全是Java程序员就别选Go了——培训成本太高
性能要求	高频交易 → C++；一般Web → Python也够
开发效率	Python开发快，C++开发慢但跑得快
平台兼容	跨平台 → Java/Kotlin/Dart/React Native
社区和库	不想重新发明轮子——选生态系统丰富的语言

小红团队选了：后端用**Java（Spring Boot）** + 前端用**React Native**（一套代码跑iOS和Android）。数据库用**MySQL**，缓存用**Redis**。

4.1.2 编码风格：让你的代码像诗一样可读

任何傻瓜都能写出计算机能理解的代码。只有好程序员才能写出人类能理解的代码。——Martin Fowler

编码风格包括三个方面：

命名规范

```
// 坏的命名：
int a;           // a是什么？鬼知道
String s;        // s又是什么？
void f(int x);   // f.....这个函数干嘛的？

// 好的命名：
int parkingSpotCount;    // 一眼就知道是车位数量
String userPhoneNumber;  // 用户手机号
void reserveParkingSpot(int spotId); // 预约车位
```

命名规则（考试重点）：

- **骆驼命名法（CamelCase）** ——Java方法名：`calculateParkingFee()`
- **帕斯卡命名法（PascalCase）** ——Java类名：`ParkingSpotManager`
- **蛇形命名法（snake_case）** ——Python变量：`parking_spot_count`
- **匈牙利命名法**——在变量名前加类型前缀（不太流行了）：`iCount`表示int

注释规范

- **为什么**写这个代码（动机），而不是**做了什么**（代码本身已经说明了做什么）
- 好的注释：`// 夜间时段(20:00-8:00)的计费规则与白天不同，详见需求文档V2.3`
- 坏的注释：`// 给x赋值5`（x = 5已经说明了）

代码布局

- 缩进统一（4个空格或1个Tab——选哪个不重要，**统一**最重要）
- 括号风格一致（K&R风格 vs Allman风格）
- 适当使用空行分隔逻辑块
- 一行不超过80-120个字符

```
// 好的代码布局：
public float calculateFee(int hours, boolean isVIP) {
    float rate = isVIP ? VIP_RATE : NORMAL_RATE;

    float fee = hours * rate;

    if (hours > 24) {
        fee = fee * LONG_STAY_DISCOUNT;
    }

    return min(fee, DAILY_CAP);
}
```

4.1.3 结构化程序设计

结构化程序设计的核心思想——**只用三种控制结构写出任何程序**：

1. **顺序结构**——一条一条执行
2. **选择结构**——if-else、switch
3. **循环结构**——while、for

不要使用GOTO！——结构化程序设计的铁律。

Edsger Dijkstra在1968年发表了一篇著名的论文《GOTO语句是有害的》（Go To Statement Considered Harmful），在计算机科学界引发了一场大辩论。今天，不使用GOTO已经成为几乎所有编程语言的共识。

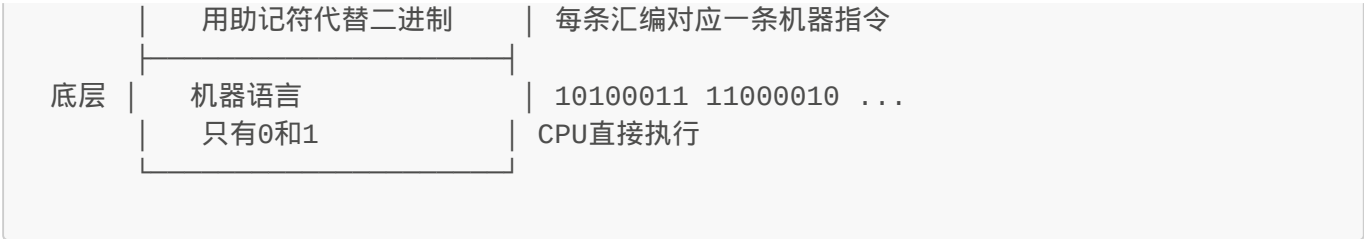
结构化程序设计的好处：

- 程序逻辑清晰，易于理解和维护
- 按自顶向下逐步细化的方式开发
- 每个代码块只有一个入口和一个出口

4.2 编程语言：从0和1到高级语言

4.2.1 语言的三个层次

高层	高级语言 接近人类自然语言	Java, Python, C++, JavaScript, Go... "if(user.isLoggedIn()) { showPage(); }"
	汇编语言	MOV, ADD, JMP, PUSH...



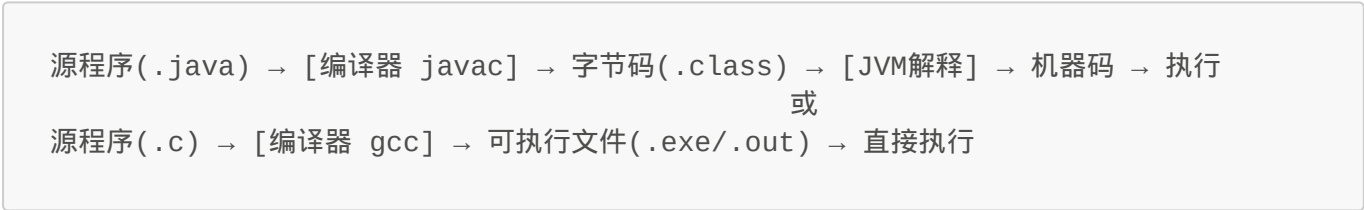
层次	特点	可读性	可移植性	执行效率
机器语言	0/1组成，CPU直接执行	极差	不（指令集相关）	最高
汇编语言	助记符代替二进制	差	不	很高
高级语言	接近自然语言	好	好（编译器决定）	取决于编译器

4.2.2 编译 vs 解释

高级语言写了不能直接跑，要变成机器语言才行。怎么变？两种方式：

编译（Compilation）

翻译整本书再读——先把整个源程序一次性翻译成目标代码（机器语言），然后执行。



优点：执行速度快（一遍翻译，反复执行） **缺点：**开发调试慢（每次改了都要重新编译） **代表语言：**C、C++、Go、Rust

解释（Interpretation）

同声传译——读一行，翻译一行，执行一行。不生成独立的目标文件。



优点：开发调试快，改代码立刻就能跑 **缺点：**执行速度慢（每次执行都重新翻译） **代表语言：**Python、JavaScript、Ruby、PHP

Java的"半编译半解释"

Java的独特设计：

- 1. 先编译：`.java` → `javac` → `.class`（字节码，与平台无关）
- 2. 后解释/即时编译：`.class` → `JVM`（Java虚拟机） → 逐条解释或即时编译（JIT）为机器码

"Write Once, Run Anywhere"——一次编写，到处运行。

[Diagram: Java的编译-解释流程。javac编译器读.java文件生成.class字节码，不同操作系统的JVM读取相同的.class文件，各自解释给各自的CPU。]

4.2.3 常用编程语言特点（考试重点）

语言	类型	主要特点	适用场景
C	编译型	高效、底层、指针	操作系统、嵌入式
C++	编译型	C + 面向对象、多范式	游戏、图形、高频交易
Java	编译+解释（JVM）	跨平台、OOP、强类型、GC	企业应用、Android、大数据
Python	解释型	易学、动态类型、库丰富	AI/ML、数据分析、Web、自动化
JavaScript	解释型（JIT）	Web标配、事件驱动、动态类型	前端、Node.js后端
Go	编译型	简洁、并发、静态类型	云原生、微服务、CLI工具
SQL	声明型	数据库查询的非过程化语言	数据库操作

GEEK NOTE

考试中关于编程语言的考题通常简单——"以下哪种语言是编译型语言"、"Java的特点是"、"解释型语言的优点是"等。记住分类和代表性语言就行了。

SHARPEN YOUR PENCIL

判断以下说法的正误：

- 1. () 机器语言的执行效率比高级语言高，因为它不需要编译或解释。
- 2. () Java是纯编译型语言。
- 3. () 解释型语言的优点之一是开发调试更加便捷。
- 4. () 结构化程序设计鼓励使用GOTO语句。
- 5. () 好的代码注释应该说明"做了什么"而不是"为什么这样做"。

(答案见本章末尾)

问与答（There Are No Dumb Questions）

Q：小红团队为什么选Java做后端？ A：因为小刚擅长Java，Java生态成熟（Spring框架、Maven管理），而且有大量现成的库做支付对接、短信验证等。选语言的第一原则永远是：团队会什么就用什么。

Q：注释写越多越好吗？ A：不是！好的代码应该**自注释**——变量名和函数名足够清晰，不需要额外说明。注释是"必要时"的补充，不是凑字数的任务。

Q：Python和Java到底哪个好？ A：这就像问"锤子和螺丝刀哪个好"——看你要干什么。Python开发快，适合原型和AI；Java性能好、类型安全，适合大型企业系统。小红先用Python做了原型验证，正式开发用了Java——两种都用！

BULLET POINTS

- 编码是把详细设计转化为可执行程序的过程
- 选语言的考量：项目类型、团队技能、性能、平台、生态
- 好的编码风格三要素：命名规范、注释（为什么而非做什么）、代码布局
- 四种命名法：骆驼（CamelCase）、帕斯卡（PascalCase）、蛇形（snake_case）、匈牙利
- 结构化程序只有三种控制结构：顺序、选择、循环。不用GOTO！
- 语言三层次：机器语言（0/1）→ 汇编语言（助记符）→ 高级语言（接近人类语言）
- 编译=翻译整本书再读（C/C++/Go）；解释=同声传译边读边译（Python/JS）
- Java是编译+解释的混合：先编译成字节码（.class），JVM再解释执行
- "Write Once, Run Anywhere"是Java的核心优势

SHARPEN YOUR PENCIL 答案：

1. √ 2. ×（Java先编译后解释） 3. √ 4. ×（不用GOTO） 5. ×（应该说明"为什么"）

第5章 软件测试：找出藏在角落里的Bug

小红的停车App终于有了第一个可运行的版本。小刚骄傲地把手机递给小红："你看，搜车位、预约、支付，全都能跑！"

小红点了一下"支付"按钮——App闪退了。

又点了一下——这次没闪退，但扣了她两次钱。

小刚的脸绿了。

"看来，能跑和能用，中间隔了1000个Bug。"小红平静地说。

5.1 软件测试基础：测试不是找"程序是对的"的证据

什么是测试？

很多人以为测试的目的是证明程序是对的——**大错特错！**

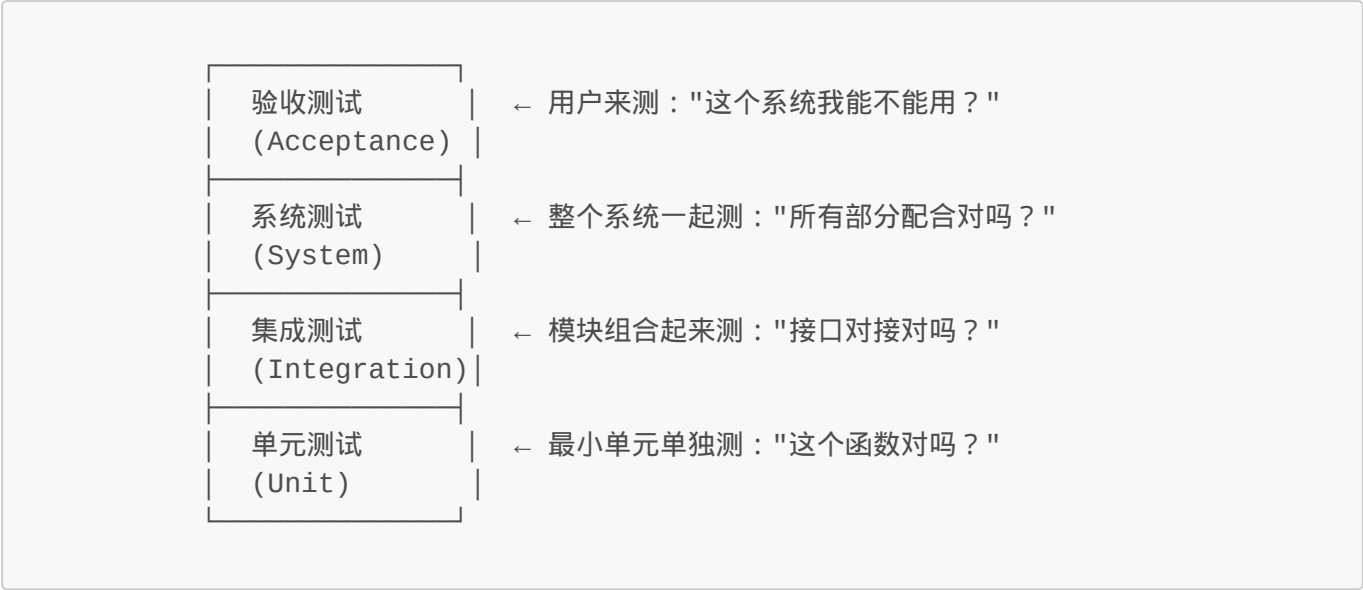
测试的目的是发现错误。一个好的测试就是找到还没有被发现的错误。

程序测试只能证明错误的存在，不能证明错误的不存在。——Edsger Dijkstra

测试的基本原则（考试必考！）

1. **尽早测试**——测试不是最后一步，要从需求分析阶段就开始
2. **测试不能由程序员自己做自己的代码**——心理盲区，自己的错误自己看不到
3. **测试用例应包含输入数据和预期输出**——没有预期输出的测试是"盲测"
4. **既要包含合理输入，也要包含不合理输入**——边界、异常、非法值
5. **全面检查测试结果**——不只是看输出对不对，还要看有没有"连带伤害"
6. **保存测试用例**——复用、回归测试

测试的四个层次



层次	测试对象	谁来做	关注点
单元测试	一个函数/类/方法	程序员自己	这个函数在给定输入下返回对吗？
集成测试	一组模块的接口	开发/测试	模块A传给模块B的数据对吗？
系统测试	整个软件系统	测试团队	全功能、性能、安全性
验收测试	可交付的系统	用户/客户	满足需求吗？可以上线吗？

WATCH IT!

单元测试和集成测试经常被混淆。区分口诀："单元测函数，集成测接口"。单元测试是一个个函数单独测（所以要用Mock来隔离其他部分），集成测试是多个模块连起来测。

测试驱动开发（TDD）再回顾

在第一章敏捷开发中我们提过TDD的Red-Green-Refactor循环。这里补充TDD的具体步骤：

1. RED：写一个测试用例，运行它 → 失败（因为功能还没写）
 2. GREEN：写最少的代码让测试通过 → 测试通过
 3. REFACTOR：优化代码结构 → 测试仍然通过
 4. 重复.....

TDD的核心价值：不是"先测试"，而是**用测试来驱动设计**。先写测试迫使你思考"这个方法应该怎么被调用"、"输入输出是什么"——这自然而然地产生了更好的接口设计。

SHARPEN YOUR PENCIL

小红团队写了一个计算停车费的函数：`calculateFee(hours, isVIP, isHoliday)`。请为这个函数设计5个单元测试用例，包括正常情况和异常情况。

(格式：输入 (hours, isVIP, isHoliday) → 预期输出)

(答案见本章末尾)

5.2 测试方法：两大类，各有千秋

静态测试 (Static Testing)

不运行程序就检查代码——用眼睛看，用脑子想。

方法包括：

- **代码审查 (Code Review)** ——同事一行一行看你的代码找问题
- **走查 (Walkthrough)** ——作者带着团队"走一遍"代码逻辑
- **静态分析工具**——用工具自动扫描 (如SonarQube、Checkstyle)

优点：不需要可运行的代码，早期就能发现问题，比动态测试便宜。

动态测试 (Dynamic Testing)

运行程序，给它输入，看输出。

动态测试分为两类：

白盒测试 (White-Box Testing)

你能看到代码内部结构，基于代码逻辑设计测试用例。像医生给病人做全身CT——什么都看得见。

黑盒测试 (Black-Box Testing)

你不看代码，只看功能——给它输入，看输出对不对。像你测试一个微波炉——你不需要知道磁控管怎么工作，你只需要知道"放进食物按2分钟，食物变热了"。

维度	白盒测试	黑盒测试
视角	内部结构可见	只看外部功能
依据	代码逻辑	需求规格说明书 (SRS)
谁来做	程序员	测试人员/用户
目标	覆盖所有代码路径	覆盖所有功能场景

5.3 白盒测试：把代码解剖了看

逻辑覆盖标准 (从弱到强)

考试中的白盒测试围绕这几个"覆盖"展开：

1. 语句覆盖 (Statement Coverage)

每一行代码至少执行一次。最弱的覆盖标准。

```
if (a > 0) {           // 行1
    x = x + 1;         // 行2
}                     // 行3
x = x + 2;             // 行4
```

测试用例： $a = 1$ （执行行1→行2→行3→行4，即行1、行2、行4都执行了。行3不是可执行语句）

问题：如果 $a=1$ ，条件 $a>0$ 为真，行2执行了。但如果 $a\leq 0$ ，行2不会执行。所以语句覆盖只要求每行至少被一条用例执行一次。

2. 判定覆盖 / 分支覆盖 (Decision/Branch Coverage)

每个判断的真假分支都执行一次。

上面例子需要两个测试用例： $a = 1$ （真分支）和 $a = 0$ （假分支）。

3. 条件覆盖 (Condition Coverage)

每个条件的每种可能取值至少出现一次。

```
if (a > 0 && b > 0) {
    x = x + 1;
}
```

条件： $a > 0$ 和 $b > 0$ 。每个条件需要取真和假各一次。

测试用例： $a=1, b=1$ （两个条件都真）、 $a=-1, b=-1$ （两个条件都假）。——覆盖了每个条件的真和假，但整个判断只走了"都真→真"和"都假→假"两个分支。

4. 判定-条件覆盖 (Decision/Condition Coverage)

同时满足判定覆盖和条件覆盖。

即：每个判断的真假都走到，且每个条件的每种取值都出现。

5. 条件组合覆盖

每个判断中所有条件取值的每种可能组合至少出现一次。

上面的 $a > 0 \ \&\& \ b > 0$ 有4种组合：

- $a=1, b=1$ (T,T)
- $a=1, b=-1$ (T,F)
- $a=-1, b=1$ (F,T)
- $a=-1, b=-1$ (F,F)

6. 路径覆盖 (Path Coverage)

程序中的每条可能执行路径都走到。最强的覆盖标准。

覆盖标准强度排序（弱→强）：

语句覆盖 → 判定覆盖 → 条件覆盖 → 判定-条件覆盖 → 条件组合覆盖 → 路径覆盖

WATCH IT!

考试中给你一段代码，让你设计测试用例满足"判定覆盖"或"条件覆盖"——这是经典题型。记住：判定覆盖看"分支"，条件覆盖看"条件取值"。先搞清楚有几个判断和几个条件，再设计用例。

基本路径测试（Basis Path Testing）

这是一种基于**圈复杂度（Cyclomatic Complexity）**的白盒测试方法。

步骤：

1. 画出程序的控制流图（Flow Graph）
2. 计算圈复杂度 $V(G)$
3. 确定独立路径集合
4. 为每条独立路径设计测试用例

圈复杂度计算（三种方法）

$$\begin{aligned} V(G) &= E - N + 2 && (E=\text{边数}, N=\text{节点数}) \\ V(G) &= P + 1 && (P=\text{判断节点数}) \\ V(G) &= \text{区域数} && (\text{控制流图平面划分的区域数}) \end{aligned}$$

示例：一个简单的if-else程序

[Diagram: 控制流图。节点1(开始) → 节点2(判断 $a > 0$) → 真分支节点3(操作A) → 节点5(结束)；假分支节点4(操作B) → 节点5(结束)。]

$E=5$ 条边， $N=5$ 个节点。 $V(G)=5-5+2=2$ 。即有2条独立路径。 $P=1$ 个判断节点。 $V(G)=1+1=2$ 。验证一致。

2条独立路径：路径1：(1 → 2 → 3 → 5)，路径2：(1 → 2 → 4 → 5)

为每条路径设计一个能走这条路径的测试用例即可。

5.4 黑盒测试：不看里面，只管外面

黑盒测试依据**需求规格说明书（SRS）**，不关心代码内部。

等价类划分（Equivalence Partitioning）

把输入数据划分为**若干等价类**，每个等价类中的数据"等效"——如果其中一个数据测出来没问题，这个类中的其他数据应该也没问题。

两种等价类：

- **有效等价类**——符合要求的、合理的输入
- **无效等价类**——不符合要求的、错误的输入

小红App的例子——输入"停车时长"：

等价类	类型	代表值	预期结果
1-24小时	有效	3	按正常费率计算
0小时	无效（下边界外）	0	错误提示"时长不能为0"
负数	无效	-1	错误提示"时长不能为负"
超过24小时	有效（但在边界上）	25	按长时间费率计算

边界值分析（Boundary Value Analysis）

Bug喜欢藏在**边界上**——这是无数程序员的血泪教训。

对每个等价类的边界，测试**边界值本身**和**边界值的两侧**。

如果"有效停车时长为1-24小时"，边界值应该是：

- 0（边界下方）
- 1（边界）
- 2（边界上方）
- 23（下边界上方）
- 24（上边界）
- 25（上方）

通常每个边界测三个值：边界值、边界值-ε、边界值+ε。

GEEK NOTE

等价类划分和边界值分析经常配合使用。先用等价类划分把输入空间分成几块，再用边界值分析对每块的边界做重点测试。考试中经常问"给一个输入范围，设计测试用例"——这时要把两个方法结合起来。

因果图法（Cause-Effect Graphing）

当输入条件之间有复杂的逻辑关系时，用因果图来系统地生成测试用例。

步骤：列出所有"因"（输入条件） → 列出所有"果"（输出/动作） → 画出因果关系 → 转换为判定表 → 生成测试用例。

适用场景：输入条件多且之间有依赖关系（比如"VIP用户且周末时段有特殊折扣"）。

正交试验法

当有多个因素、每个因素有多个取值时，全排列测试太多，用正交表选出**代表性的组合**来测试。

错误推测法

基于测试人员的经验和直觉，猜测哪里可能出错，然后设计测试用例。

常见错误推测目标：空值、零值、特殊字符、并发操作、网络中断.....

5.5 测试策略：怎么把测试组织起来？

集成测试策略

当你把各个模块写好，要合在一起测的时候，有三种策略：

自顶向下集成（Top-Down Integration）

从顶层模块开始，逐步集成下层模块。下层没有的用**桩模块（Stub）**代替。

第1步：测主控模块（底层模块用Stub代替）
第2步：集成一级子模块（替换Stub）
第3步：集成二级子模块.....

优点：尽早发现顶层设计错误 缺点：需要大量Stub，底层模块很晚才被测试

自底向上集成（Bottom-Up Integration）

从底层模块开始，逐步集成上层。上层没有的用**驱动模块（Driver）**调用底层。

优点：底层测试充分，不需要Stub 缺点：顶层模块很晚才被测，整体错误发现得晚

三明治 / 混合集成

上层用自顶向下，底层用自底向上，中间某处汇合。结合了两者的优点。



回归测试（Regression Testing）

你改了一个Bug，然后测试所有之前正常的功能是否依然正常。

因为软件世界有一个残酷的规律：修复引入的Bug往往比修复的Bug更可怕。

自动化回归测试是现代软件开发的标配——每次提交代码都自动跑一遍全部测试用例。

α测试与β测试

当系统基本完成，在交付给最终用户之前：

测试	场所	测试者	特点
α测试（Alpha）	开发方场所	用户代表 + 开发者	开发者可观察、可控
β测试（Beta）	用户实际场所	真实用户	开发者不可控，收集真实反馈

"小红的停车App，先在团队内部做α测试（找5个朋友试），然后做一个β测试版放出去给100个种子用户试用。"

自动化测试

用工具代替人工执行测试。常见类型：

- **单元测试自动化**——JUnit (Java)、pytest (Python)
- **UI自动化测试**——Selenium、Playwright
- **API自动化测试**——Postman、JMeter
- **持续集成（CI）** ——每次提交跑全量测试 (Jenkins、GitHub Actions)

BE THE ANALYST：为停车App设计测试

小红团队完成了"预约车位"模块。代码如下（简化版）：

```
function reserve(userId, spotId, hours):
    if (userId <= 0) return "无效用户"
    if (spotId <= 0) return "无效车位"
    if (hours < 1 or hours > 24) return "时长超出范围"

    spot = getSpot(spotId)
    if (spot.status == "占用") return "车位已被占用"
    if (spot.status == "维护") return "车位维护中"

    createReservation(userId, spotId, hours)
    return "预约成功"
```

请完成以下任务：

- 任务1（白盒测试）：**画出控制流图，计算圈复杂度，列出所有独立路径。
- 任务2（黑盒测试）：**用等价类划分和边界值分析方法，设计hours参数的测试用例。
- 任务3（综合）：**设计5个测试用例，要求同时包含正常和异常情况，每个用例说明输入和预期输出。
- （参考答案见本章末尾）

问与答（There Are No Dumb Questions）

Q：白盒和黑盒测试哪个更好？ A：没有谁更好，它们互补。白盒确保代码逻辑不遗漏，黑盒确保功能不遗漏。就像检查一辆车：白盒是打开发动机盖看，黑盒是开一圈试驾。

Q：语句覆盖和路径覆盖有什么区别？ A：语句覆盖要求每行至少执行一次。路径覆盖要求每种可能的执行路径都走一遍。比如if-else嵌套，语句覆盖可能两条路径就覆盖了所有语句，但路径覆盖可能要4条（2×2）。

Q：什么时候用自顶向下，什么时候用自底向上？ A：自顶向下适合当整体架构最重要的时候（能尽早验证架构），自底向上适合当底层模块是关键的时候（比如底层算法需要非常可靠的测试）。

Q：TDD是不是就一定能写出没有Bug的代码？ A：不能。TDD能大幅减少低级Bug，但不能替代系统测试和验收测试。TDD主要是设计工具，不只是测试工具。

BULLET POINTS

- 测试的目的是发现错误，不是证明程序正确！
- 测试六大原则：尽早测试、自己不能测自己、含预期输出、合理+不合理输入、全面检查、保存用例
- 测试四层次（从细到粗）：单元→集成→系统→验收
- 静态测试（不运行：代码审查、走查） vs 动态测试（运行：白盒+黑盒）
- 白盒测试逻辑覆盖（弱→强）：语句覆盖<判定覆盖<条件覆盖<判定-条件覆盖<条件组合覆盖<路径覆盖
- 圈复杂度 $V(G) = E - N + 2 = P + 1 = \text{区域数}$ 。用于基本路径测试
- 黑盒测试五种方法：等价类划分、边界值分析、因果图法、正交试验法、错误推测法
- 等价类分为有效和无效。边界值通常测：边界-1、边界、边界+1
- 集成策略三种：自顶向下（用Stub）、自底向上（用Driver）、三明治（混合）
- 回归测试：改了代码后重新测，确保没引入新Bug
- α 测试在开发方（可控）， β 测试在用户方（真实环境）
- TDD（Red → Green → Refactor）不仅用于测试，更用于驱动设计

SHARPEN YOUR PENCIL 答案（calculateFee测试用例）：

1. (1小时, false, false) → 预期：8元（非会员白天）
2. (1小时, true, false) → 预期：5元（会员白天）
3. (0小时, true, false) → 预期：0元或错误提示（边界：零值）
4. (1小时, false, true) → 预期：按节假日费率计算（边界：节假日）
5. (25小时, false, false) → 预期：按长时间阶梯费率计算（边界：超过24小时）

第6章 软件维护：产品上线了，但故事才刚开始

小红的停车App终于正式上线了。第一天就拿到了500个下载，7个好评，2条投诉。

投诉1："预约了车位结果去了发现被占了！" 投诉2："iOS 18更新后App闪退！" 投诉3："什么时候能加一个'帮我记住常去停车场'的功能？"

小红看着这些反馈，突然意识到：

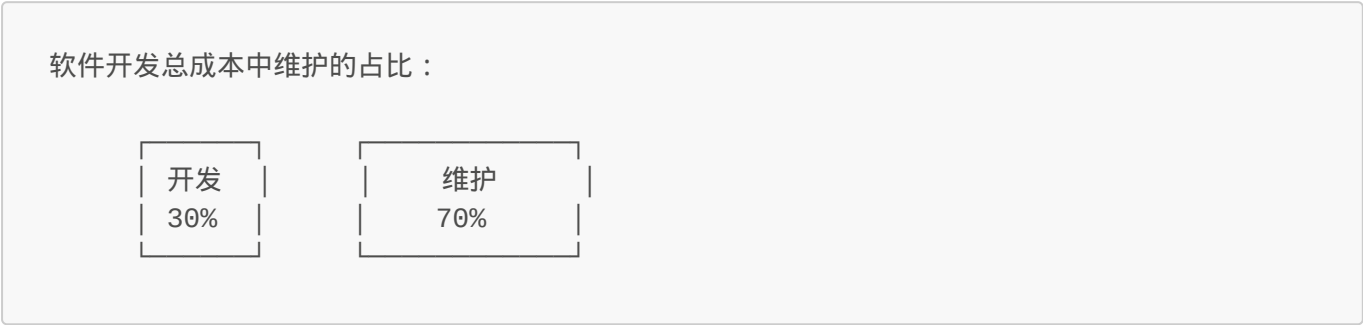
软件上线不是结束，而是另一个漫长旅程的开始。

6.1 软件维护基本概念

什么是软件维护？

软件维护是在**软件已经交付使用之后**，为了改正错误、适应环境变化或改进性能而修改软件的过程。

在整个软件生命周期中，维护占据了**50%-70%的成本**。是的，你没有看错——维护比开发贵得多。



[Diagram: 饼图。一个大圆，大约30%是"开发"，70%是"维护"。旁边配字："你不是在开发新功能，就是在修旧功能。"]

软件维护的四种类型（考试必考）

类型	英文	占比	说明	小红的例子
改正性维护	Corrective	~17-21%	修Bug——用户发现的错误	"预约了车位结果被占了"
适应性维护	Adaptive	~18-25%	适应外部环境变化	"iOS 18更新后闪退"
完善性维护	Perfective	~50-55%	新增功能、改进性能	"加一个记住常去车位的功能"
预防性维护	Preventive	~4-5%	预防未来可能出现的问题	重构旧代码，加更好的日志

WATCH IT!

四种维护的占比是考试重点！**完善性维护占比最大（超过一半！）**，预防性维护占比最小。记住口诀："改正适应完善防，完善过半预防少。"

改正性维护 vs 适应性维护的区别（高频易错点）：

- **改正性**：软件有Bug——系统本身的问题（内部原因）
- **适应性**：环境变了——系统没问题，是外部变了（外部原因）

比如："App在Android 14能跑，Android 15闪退" → **适应性维护**（操作系统变了） "App在搜索车位时偶尔崩溃" → **改正性维护**（代码有Bug）

软件维护的过程

维护不是"打开代码就改"，它是一个有组织的过程：

1. **维护请求/问题报告**——用户提交反馈或Bug报告
2. **分类与优先级确定**——这是改正性还是完善性？紧急还是不紧急？

3. **分析与理解**——弄清楚要改什么、影响范围多大
4. **修改设计与实现**——做出修改
5. **回归测试**——确保改了对、没引入新Bug
6. **发布与文档更新**——发布新版本，更新用户手册和设计文档

SHARPEN YOUR PENCIL

请判断以下维护场景属于哪种类型：

1. 用户反映App在某些手机上字体显示不全，经排查是屏幕适配问题。_____
2. 支付宝改了支付接口的API签名方式，停车App需要跟着改。_____
3. 用户建议加一个"拼车共享车位"功能，团队决定下个版本加入。_____
4. 团队发现数据库表结构设计不合理，主动重构——虽然现在能跑，但预感数据量大了会崩。_____
5. 用户投诉"点了支付扣了钱但没生成订单"。_____

(答案见本章末尾)

6.2 可维护性：让软件不容易"生锈"

什么是可维护性？

可维护性是指软件被**理解、改正、适应和增强**的容易程度。

一个可维护性高的软件像一个乐高模型——哪里坏了换哪里，想加新东西往上面拼就行。一个可维护性低的软件像一团揉在一起的橡皮泥——动一个地方不知道会影响多少其他地方。

影响可维护性的因素（考试重点）

因素	说明
可理解性	别人能看懂你的代码吗？命名、注释、结构是否清晰？
可测试性	改了代码以后容易测试吗？有自动化测试吗？
可修改性	改一个地方不影响其他地方吗？（低耦合高内聚）
可靠性	程序出错后有清晰的错误信息和日志吗？
可移植性	换一个环境跑容易吗？
可使用性	运行效率能满足用户需求吗？

提高可维护性的方法

1. **严格遵循编码规范**——统一命名、格式、注释
2. **模块化设计（高内聚低耦合）**——每个模块功能单一
3. **完整的文档**——需求文档、设计文档、用户手册、维护手册
4. **使用版本控制**——Git记录每一次修改
5. **建立测试体系**——自动化回归测试
6. **定期重构**——保持代码"健康"

GEEK NOTE

提高可维护性的方法中，"模块化"和"文档"是考试最爱考的两个。它们分别对应技术层面和管理层面。
记住：**可维护性不是天生的，是设计和纪律的结果。**

6.3 软件再工程（Software Reengineering）

有时候，软件老到一定程度，不是修修补补能解决的问题了。这时候就需要"再工程"——给软件来一次大手术。

再工程包含三个层次：

逆向工程（Reverse Engineering）

从**代码反推设计**的过程。就像你做一道菜，但菜谱丢了，你尝了一口试图反推出来里面放了什么。

代码 → [分析] → 设计模型 → [分析] → 需求模型

为什么要逆向工程？

- 接手一个没有文档的老系统（"遗留系统"）
- 原开发团队已经走光了
- 文档已经过时了

逆向工程不是"破解"或"解密"——它是**理解已有代码的高层结构**。

重构（Refactoring / 再构）

不改变系统外部行为，只改善内部结构。

重构前：	重构后：
<pre>public double f(double h, boolean v) { double r = v ? 5 : 8; STANDARD_RATE; return h * r; }</pre>	<pre>public double calculateParkingFee(double hours, boolean isVIP) { double rate = isVIP ? VIP_RATE : return hours * rate; }</pre>

重构的关键原则：**小步快跑**——每次只做一个小的改进，然后跑测试确保没有破坏功能。

正向工程（Forward Engineering）

从**设计模型重新生成代码**。通常是逆向工程师的后续——先逆向理解老系统，再正向构建新系统。

需求模型 → [设计] → 设计模型 → [实现] → 新代码

三者的关系：

老系统代码 —[逆向工程]—> 设计模型 —[重构]—> 改进的设计模型 —[正向工程]—> 新系统代码

RELAX

再工程的三个概念考试中通常以选择题出现——给你一个场景，问属于哪种工程。逆向→从代码到设计（倒推），重构→改内部不改外部（装修），正向→从设计到代码（新建）。

问与答（There Are No Dumb Questions）

Q：为什么维护成本这么高？ A：因为维护不是"改一行代码"那么简单。你需要理解原代码（可能是几年前别人写的或者自己写的但早已忘了）、分析改动的影响范围、修改后全面回归测试、更新文档……每一步都可能很耗时。

Q：适应性维护和完善性维护有什么区别？ A：适应性是"被动"的——被外部逼的（系统升级、法律变了）。完善性是"主动"的——用户想要更好/更多的功能。

Q：重构和改写（Rewrite）是一回事吗？ A：不是！重构是"装修"——在现有结构上改善。Rewrite是"推倒重建"——完全重新写。重构更安全、成本更低，但有时候系统的"腐烂"程度太深，可能不得不Rewrite。

Q：预防性维护有必要吗？ A：有！就像你每年做体检——现在花点小钱，防止以后花大钱。预防性维护占比最小（~4-5%）但长期来看ROI很高。

BULLET POINTS

- 软件维护占整个软件成本的50%-70%
- 维护的四种类型：改正性（修Bug, ~17-21%）、适应性（适应环境变化, ~18-25%）、完善性（加功能/改进, ~50-55%，最多！）、预防性（防患未然, ~4-5%）
- 改正性=内部Bug，适应性=外部环境变化——这是区分的关键
- 维护过程：请求→分类→分析→修改+回归测试→发布+更新文档
- 可维护性六大影响因素：可理解性、可测试性、可修改性、可靠性、可移植性、可使用性
- 提高可维护性：编码规范、模块化、完整文档、版本控制、自动化测试、定期重构
- 软件再工程三层次：
 - 逆向工程：代码→设计（倒推理解）
 - 重构：不改外部行为，优化内部结构
 - 正向工程：设计→代码（重新实现）

SHARPEN YOUR PENCIL 答案（维护类型）：

1. 改正性维护（适配问题是开发时没考虑到的Bug）
2. 适应性维护（外部API变了，代码需要适应）
3. 完善性维护（主动加新功能，占维护最大比例！）
4. 预防性维护（防患未然，还没出问题但主动优化）
5. 改正性维护（典型的Bug：功能有缺陷）

第7章 软件项目管理：小红的团队越来越大了

停车App用户突破10万。小红从3人小团队发展到15人的正规军。

然后问题来了。

"老板，这个功能要做多久？" "老板，我们是不是超预算了？" "老板，上次改的代码和这次改的冲突了，系统挂了！" "老板，万一支付系统宕机怎么办？"

小红终于理解了：**写代码只是软件工程的冰山一角。管理一个软件项目，是整座冰山。**

7.1 软件项目计划：算清楚再出发

7.1.1 软件规模估算

在你开始写一行代码之前，你就要回答一个问题：**这个项目有多大？**

代码行（Lines of Code, LOC）

最直观但也最粗糙的方法——用预期的代码行数来衡量软件规模。

优点：简单直观 **缺点：**

- 不同语言不能比（Java一行和Python一行做的事差别很大）
- 鼓励程序员写冗长的代码（"我的绩效看代码行数？那我多写几行"）
- 在项目开始时很难预估

功能点（Function Points, FP）

比LOC更科学的方法——**数功能，不数代码行。**

功能点方法关注软件向用户提供的"功能"，分为五类：

功能类型	含义	小红的App例子
外部输入（EI）	用户向系统输入的数据	搜索条件、注册信息
外部输出（EO）	系统向用户输出的数据	车位列表、支付凭证
外部查询（EQ）	用户查询，系统即时响应	查历史订单、查余额
内部逻辑文件（ILF）	系统内部维护的数据	用户表、车位表、订单表
外部接口文件（EIF）	与其他系统交互的数据	支付网关接口、短信接口

每种类型根据复杂度（简单/中等/复杂）赋予不同权重，最后加权求和得到总功能点数。

优点：与语言无关、与实现无关、更客观 **缺点：**计算比较复杂，有一定主观性

考试通常考LOC和FP的概念和优缺点对比。记住：LOC简单但不科学（语言相关），FP更科学（功能导向）但计算复杂。

7.1.2 工作量估算：COCOMO模型

知道了规模（多少行代码/多少个功能点），下一步是估算**需要多少人干多久**。

COCOMO（Constructive Cost Model，构造性成本模型）由Barry Boehm在1981年提出，是软件工程中最经典的工作量估算模型。

COCOMO分三个层次：

层次	说明	适用阶段
基本COCOMO	仅用代码行数估算	项目早期
中间COCOMO	加入15个成本驱动因子	详细设计后
详细COCOMO	按阶段分别估算	各阶段完成后

基本COCOMO公式：

工作量 $E = a \times (KLOC)^b$ （人月）

开发时间 $D = c \times E^d$ （月）

其中KLOC是千行代码数。a、b、c、d根据项目类型不同：

项目类型	说明	a	b
组织型（Organic）	小团队、熟悉领域、需求稳定	2.4	1.05
半独立型（Semi-detached）	中等规模、混合团队	3.0	1.12
嵌入型（Embedded）	约束严格、高风险	3.6	1.20

例子：小红停车App预计3万行代码（30KLOC），属于组织型项目。估算工作量： $E = 2.4 \times (30)^{1.05} \approx 2.4 \times 35.9 \approx \mathbf{86人月}$ 。如果3个人开发，大约需要29个月——"呃，这不太对啊....."

当然，COCOMO只是一个估算起点，实际情况要不断调整。

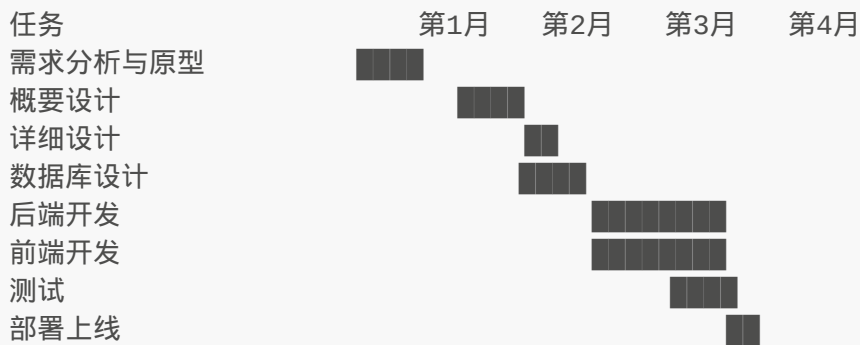
7.1.3 进度计划

估算出工作量和时间后，还要做详细的进度计划。

甘特图（Gantt Chart）

最直观的项目进度工具——横轴是时间，纵轴是任务，每个任务用一个横条表示。

甘特图示例——停车App第一阶段开发计划：

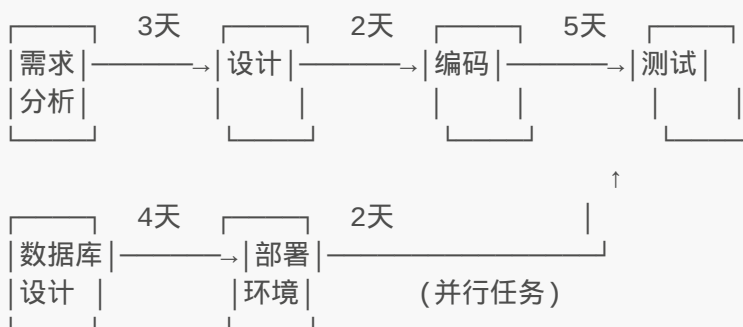


优点：直观、简单、一眼看出进度 **缺点：**看不出任务之间的**依赖关系**——哪个任务必须先完成别的任务才能开始？

PERT图 / 网络图 + 关键路径法（CPM）

PERT（Program Evaluation and Review Technique）图用网络图表示任务之间的依赖关系。

PERT图示例（简化版停车App开发任务）：



关键路径：需求分析 → 设计 → 编码 → 测试 = 3+2+5+5 = 15天

另一条路：数据库设计 → 部署环境 = 4+2 = 6天（不在关键路径上）

关键路径（Critical Path）：从项目开始到结束，**耗时最长的那条路径**。关键路径上的任务任何延迟都会导致整个项目延迟。

关键路径法（CPM）步骤：

1. 列出所有任务及其依赖关系
2. 画出网络图
3. 标注每个任务的持续时间
4. 计算每条路径的总时长
5. 最长的路径 = 关键路径

WATCH IT!

甘特图和PERT图的功能区分是选择题常客：

- **甘特图**：直观显示时间进度，但不显示依赖关系
- **PERT图/网络图**：显示任务依赖关系，可计算关键路径

项目管理中两者通常配合使用。

7.2 软件质量管理

什么是软件质量？

软件质量不是一个模糊的"好"或"不好"——它是多维度的、可度量的。

ISO 9126质量模型（考试重点）定义了软件质量的六大特征：

特征	子特征	描述
功能性（Functionality）	适合性、准确性、互操作性、安全性	软件做了该做的事吗？
可靠性（Reliability）	成熟性、容错性、易恢复性	软件稳定吗？出错后能恢复吗？
易用性（Usability）	易理解、易学、易操作	用户上手快吗？
效率（Efficiency）	时间特性、资源特性	跑得快吗？占资源多吗？
可维护性（Maintainability）	易分析、易改变、稳定性、易测试	改代码容易吗？
可移植性（Portability）	适应性、易安装、共存性、易替换	换个环境能跑吗？

口诀：功可易效维移——功能性、可靠性、易用性、效率、可维护性、可移植性。

软件质量保证（SQA）

SQA（Software Quality Assurance）不是测试——它比测试范围更广。SQA是贯穿整个软件开发过程的**质量管理活动体系**。

SQA的核心活动：

1. **制定质量标准 and 规范**——团队必须遵循的标准（编码规范、文档规范）
2. **技术评审（Review）**——对需求、设计、代码的同行评审
3. **软件测试**——动态验证
4. **配置管理**——版本控制、变更控制
5. **缺陷跟踪与管理**——Bug的记录、分派、修复、验证的完整流程
6. **度量与分析**——收集质量数据（Bug密度、测试覆盖率），持续改进

软件评审

软件评审是SQA中最重要的活动之一——**发现缺陷远比修复缺陷便宜**。

评审类型：

- **技术评审（Technical Review）**——技术专家评估技术方案和实现
- **正式审查（Formal Inspection）**——有组织的、系统化的缺陷检测过程

- **走查 (Walkthrough)** ——作者引导团队走一遍设计或代码

GEEK NOTE

评审的"1:10:100原则"：需求阶段发现一个错误花1元，设计阶段花10元，测试阶段花100元，发布后修复花1000元。越早发现问题越便宜！

7.3 软件配置管理 (SCM)

小红的团队之前只有一个Git仓库，想改就改——偶尔会有人覆盖了别人的代码，或者在错误的分支上开发。

配置管理就是来解决这个问题的。

什么是配置管理？

配置管理是用技术的和行政的手段来**标识、控制、记录和审计**软件在整个生命周期中的变化。

软件配置项 (Software Configuration Item, SCI) ——配置管理的最小单元，包括：

- 源程序代码
- 文档（需求、设计、测试、用户手册）
- 数据（数据库结构、配置文件）
- 可执行程序

版本控制 (Version Control)

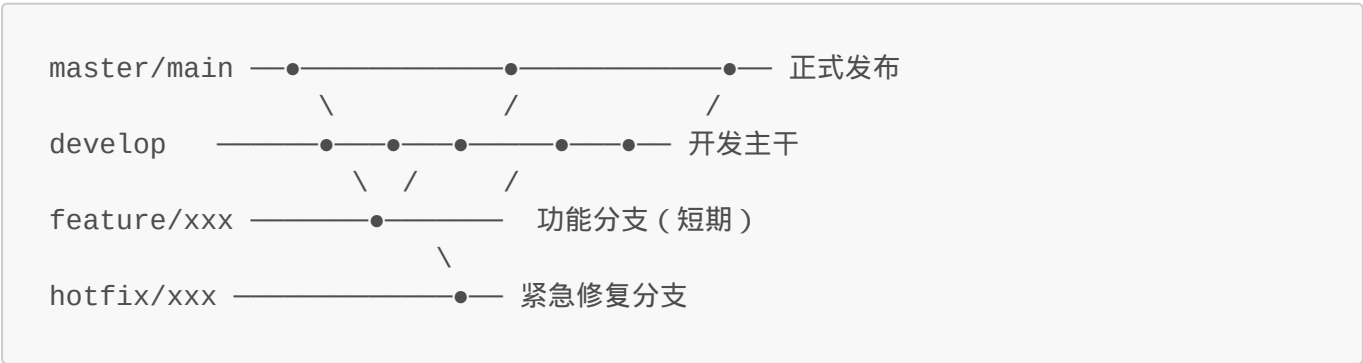
Git的基本工作流：



版本控制的关键概念：

- **版本 (Version)** ——每一次提交产生一个新的版本
- **分支 (Branch)** ——独立开发线（如feature/payment）
- **合并 (Merge)** ——把分支的修改合并回主干
- **标签 (Tag)** ——给重要版本打标记（如v1.0.0, v2.0.0）

常用分支策略——Git Flow：



变更控制

不是任何人有想法就能改代码。变更控制是一套流程：

- 1. 提交**变更请求 (Change Request)**
- 2. 变更控制委员会 (CCB) 评估变更的影响和必要性
- 3. 批准/拒绝
- 4. 如果批准：实施变更 + 测试 + 更新文档
- 5. 纳入新版本

配置审计

定期检查：代码库中的内容是否和文档描述的一致？版本号是否正确？变更是否按规定流程操作？

7.4 风险管理

小红在一次会议上问团队："我们这个项目最大的风险是什么？"

小刚："用户量太大服务器扛不住。" 小美："支付接口调用超限被罚款。" 新产品经理："竞争对手出一个免费版。"

小红把这三个写在了白板上："这三个就是我们的**Top 3 Risks**。现在，我们一个一个想办法应对。"

风险识别

用头脑风暴、专家访谈、检查表 (Checklist) 等方法，列出所有可能影响项目的风险。

常见风险分类：

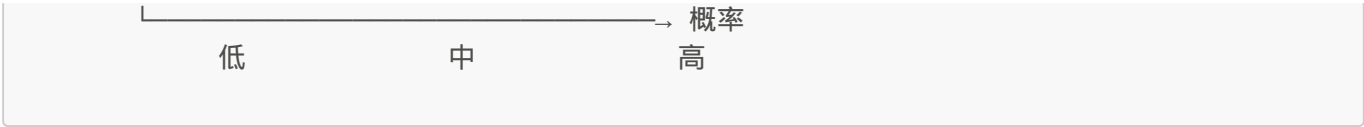
- **技术风险**——新技术不可行、性能不达标
- **人员风险**——核心程序员离职、团队技能不足
- **管理风险**——进度估算偏差、预算超支
- **外部风险**——法律政策变化、第三方服务不可用
- **需求风险**——需求频繁变更、用户不满意

风险分析

对每个已识别的风险，评估两个维度：

风险暴露量 = 风险概率 × 风险影响





风险缓解与监控

风险缓解（Risk Mitigation）——采取措施降低风险的概率或影响：

风险	缓解措施
服务器扛不住	提前做压力测试、配置自动扩容（Auto Scaling）
支付接口超限	与支付平台提前沟通扩容方案
竞争对手出免费版	差异化——我们专注服务体验和数据精准度

风险监控：定期审查风险清单，更新风险评估，检查缓解措施是否有效。

RELAX

风险管理在考试中主要是概念题——风险识别、风险分析（概率×影响）、风险缓解。记住风险矩阵和"概率×影响=风险暴露量"这个公式即可。

SHARPEN YOUR PENCIL

小红停车App的简化项目任务如下：

任务	依赖	耗时（天）
A. 需求分析	无	5
B. 数据库设计	A	3
C. 后端开发	B	10
D. 前端开发	A	8
E. 集成测试	C, D	4
F. 部署上线	E	1

1. 画出PERT网络图（用箭头表示依赖）。
2. 计算关键路径及其总时长。
3. 如果任务D（前端开发）多花了2天，会不会影响项目总工期？为什么？

（答案见本章末尾）

问与答（There Are No Dumb Questions）

Q：功能点（FP）和代码行（LOC）哪个更好？ A：功能点更客观（跟语言无关），但计算复杂。LOC简单但有语言偏见。实际项目中两者经常配合使用——先估算功能点数，再根据历史数据换算成代码行。

Q：关键路径上任务延迟了怎么办？ A：关键路径上的任何延迟都会导致项目总工期延长，所以必须重点监控。可以采取赶工（加班）或快速跟进（并行化）来压缩。非关键路径上的任务延迟只要不超过"浮动时间（Float）"，就不会影响总工期。

Q：SQA和测试是一回事吗？ A：不是！测试是SQA的一部分。SQA还包括技术评审、配置管理、规范制定、质量度量等。测试是"查Bug"，SQA是"从制度上保证少产生Bug"。

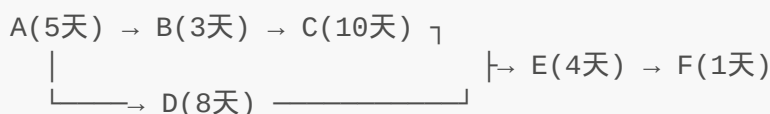
Q：Git分支太多了会不会有问题？ A：不会，分支是Git的优势。但要注意：分支应该**短命**——功能写完就合并回主干，不要长期存在的"僵尸分支"。小红团队规定：一个功能分支的生命周期不超过1个Sprint（2周）。

BULLET POINTS

- 软件规模估算：LOC（简单但有语言偏见） vs FP（客观但计算复杂）
- FP五要素：外部输入(EI)、外部输出(EO)、外部查询(EQ)、内部逻辑文件(ILF)、外部接口文件(EIF)
- COCOMO模型：基本（仅LOC）、中间（+15因子）、详细（按阶段）
- 基本COCOMO公式： $E = a \times (KLOC)^b$ 。三种项目类型参数不同
- 甘特图：直观显示进度，但不显示依赖关系
- PERT图/网络图：显示任务依赖，可计算关键路径
- 关键路径（Critical Path）= 从开始到结束最长的那条路径
- ISO 9126质量模型六大特征：功能性、可靠性、易用性、效率、可维护性、可移植性
- SQA包含：规范制定、技术评审、测试、配置管理、缺陷跟踪、度量分析
- 评审的1:10:100原则：越早发现缺陷越省钱
- 配置管理=版本控制+变更控制+配置审计
- 风险暴露量 = 风险概率 × 风险影响
- 风险管理流程：识别 → 分析 → 缓解 → 监控

SHARPEN YOUR PENCIL 答案（PERT/CPM）：

1. PERT网络图：



2. 两条路径：

- $A \rightarrow B \rightarrow C \rightarrow E \rightarrow F = 5+3+10+4+1 = 23$ 天（关键路径！）
- $A \rightarrow D \rightarrow E \rightarrow F = 5+8+4+1 = 18$ 天

3. D多花2天（从8天变10天），则 $A \rightarrow D \rightarrow E \rightarrow F = 5+10+4+1 = 20$ 天，仍然小于关键路径的23天。所以**不会影响总工期**——D有5天的浮动时间（ $23-18=5$ ）。

第5章 BE THE ANALYST 参考答案（预约车位模块测试）：

任务1（白盒测试）： 控制流图节点：1(开始) → 2(判断 $userId \leq 0$) → 3(return"无效用户")/4(判断 $spotId \leq 0$) → 5(return"无效车位")/6(判断 $hours < 1$ or > 24) → 7(return"时长超出范围")/8(getSpot) → 9(判断

status=="占用")→10(return"已被占用")/11(判断status=="维护")→12(return"维护中")/13(createReservation)→14(return"预约成功")。

圈复杂度： $V(G)=P+1=5+1=6$ （5个判断节点）。

6条独立路径：

- (1) userId无效→返回"无效用户"
- (2) spotId无效→返回"无效车位"
- (3) hours超出范围→返回"时长超出范围"
- (4) 车位已占用→返回"车位已被占用"
- (5) 车位维护中→返回"车位维护中"
- (6) 一切正常→返回"预约成功"

任务2（黑盒测试-hours参数）：有效等价类：1~24小时。边界值应测：0、1、2、23、24、25。无效等价类： ≤ 0 （测0、-1）、 ≥ 25 （测25、26）。

任务3（综合测试用例5个）：

1. reserve(100, 50, 3)→预期："预约成功"（正常路径）
2. reserve(-1, 50, 3)→预期："无效用户"（userId异常）
3. reserve(100, 0, 3)→预期："无效车位"（spotId为0）
4. reserve(100, 50, 0)→预期："时长超出范围"（hours=0边界）
5. reserve(100, 50, 25)→预期："时长超出范围"（hours=25边界）

第四篇综合 Crossword Puzzle

翻完七个章节，你的大脑是不是已经塞满了？来做个填字游戏放松一下——但别被骗了，这个"放松"会让你把学过的概念再巩固一遍！

横向（Across）

3. 软件工程中占比最大的维护类型（3字）
5. Scrum中每个开发周期叫一个_____（2字，英文音译）
8. 面向对象中表示"继承"关系的UML关系名（2字）
9. 结构化分析的核心数据流工具，缩写（3字母）
10. 软件危机五大表现之一，也是最直观的问题——_____超支（2字）
12. 在开发方场所进行的用户测试，叫什么测试（1个希腊字母）
14. 软件工程的四大目标之一，指软件容易修理和升级（4字）
15. 瀑布模型最大特点：阶段_____执行，不可逆转（2字）
17. 软件 = 程序 + 数据 + _____（2字）
18. ISO 9126质量模型六大特征中描述"跑得快吗"的那个（2字）

纵向（Down）

1. 模块之间共享全局数据区的耦合类型（4字）
2. TDD循环的三步：Red → Green → _____（重构的英文）
4. RUP四个阶段中，建立架构基线的那个阶段（2字，英文音译）
6. 测试的四个层次中，程序员自己对自己写的函数做的测试（3字，最后一个字是"试"）

[illegible]

Crossword Puzzle 答案

横向：

- 3. 完善性
- 5. 冲刺
- 8. 泛化
- 9. DFD
- 10. 预算
- 12. α (Alpha)
- 14. 可维护性
- 15. 顺序
- 17. 文档
- 18. 效率

纵向：

1. 公共耦合
2. Refactor
4. 细化
6. 单元测试
7. SRS
11. LOC
13. 个体
16. 内聚

第四篇总结：软件工程的"大局观"

恭喜你！你跟着小红从她的创业故事中走过了软件工程的完整生命周期。

让我们回顾一下这趟旅程：

小红的停车App，从最初的"我两天就能写出来"到上线后的持续维护，经历了：

问题定义 → 可行性研究 → 需求分析（第2章）

→ 总体设计 → 详细设计（第3章）

→ 编码（第4章）

→ 测试（第5章）

→ 上线维护（第6章）

整个过程被项目管理（第7章）贯穿始终

这不是一条直线，而是一个循环。真实世界的软件开发就是这样——**分析、设计、编码、测试、维护，反反复复，螺旋上升。**

最后的RELAX

软件工程这部分在考试中占约20分。题型以**选择题**和**简答题**为主，偶尔有**案例分析题**（比如给你一个场景，让你选择合适的开发模型、画出UML图、设计测试用例）。

考试重点优先级排序：

1. **稳拿分的**：软件危机、瀑布模型、敏捷宣言、RUP四阶段、耦合与内聚分类、维护四类型、ISO 9126
2. **需要理解的**：DFD/数据字典、UML用例图+类图、白盒测试（逻辑覆盖）、黑盒测试（等价类+边界值）、COCOMO、关键路径
3. **了解即可的**：设计模式23种细节、组件图/部署图、PAD图、正交试验法

本篇的**Bullet Points**已经在每章末尾为你准备好了——考试前翻一遍，比刷一百道题都管用。

记住小红的第一课，也是最重要的一课：

软件工程不是关于写代码。它是关于和一群人一起，用一种**靠谱**的方法，把正确的软件做出来，并让它一直跑下去。

——第四篇 完——